

Symphony Series

Programming Manual

Symphony5 | Symphony 10 | Symphony15



Manual Version : V1.6

Date :2024. 7. 25

PULCON

Puloon Technology Inc.

목 차

1	트리뷰 프로그램	9
1.1.	기본 명령어 설정	10
1.1.1	일반	10
1.1.2	제어	196
1.1.3	고급	204
1.1.4	플러그인	207
2	스크립트 프로그램	212
2.1.	협동로봇 스크립트 언어	212
2.1.1.	기본 프로그램 구조	212
2.1.2.	프로그램 실행 순서	214
2.1.3.	기본 문법	214
2.2.	스크립트 명령 해설 형식	240
2.3.	동작 스크립트	241
2.3.1.	<i>motion.speed</i> : 이동 속도 설정	244
2.3.2.	<i>motion.accel</i> : 가속도 설정	246
2.3.3.	<i>motion.decel</i> : 감속도 설정	248
2.3.4.	<i>joint</i> : 관절형 위치 변수 초기화	250
2.3.5.	<i>pose</i> : 직교형 위치 변수 초기화	253
2.3.6.	<i>j2p</i> : 위치 값 변환(관절에서 직교)	256
2.3.7.	<i>inv</i> : 포즈 변수의 역행렬을 직교 좌표계로 변환	258
2.3.8.	<i>movep</i> : 보간이동	261
2.3.9.	<i>movel</i> : 선형이동	266
2.3.10.	<i>movec</i> : 원호이동	272
2.3.11.	<i>moves</i> : 스플라인 곡선 이동	276
2.3.12.	<i>rmovep</i> : 상대 보간이동	280
2.3.13.	<i>rmovel</i> : 상대 선형이동	283
2.3.14.	<i>trans</i> : 병진이동	286
2.3.15.	<i>transx</i> : 병진이동 (X축 기준)	289
2.3.16.	<i>transy</i> : 병진이동 (Y축 기준)	292
2.3.17.	<i>transz</i> : 병진이동 (Z축 기준)	295
2.3.18.	<i>rot</i> : 회전이동	298
2.3.19.	<i>rotx</i> : 회전이동 (X축 기준)	302
2.3.20.	<i>roty</i> : 회전이동 (Y축 기준)	306
2.3.21.	<i>rotz</i> : 회전이동 (Z축 기준)	310
2.3.22.	<i>rotj1</i> : 회전이동 (관절 1축 기준)	314
2.3.23.	<i>rotj2</i> : 회전이동 (관절 2축 기준)	317

2.3.24.	<i>rotj3</i> : 회전이동 (관절 3축 기준).....	320
2.3.25.	<i>rotj4</i> : 회전이동 (관절 4축 기준).....	323
2.3.26.	<i>rotj5</i> : 회전이동 (관절 5축 기준).....	326
2.3.27.	<i>rotj6</i> : 회전이동 (관절 6축 기준).....	329
2.3.28.	<i>tool.trans</i> : 툴 좌표계 기준 병진이동.....	332
2.3.29.	<i>tool.transx</i> : 툴 좌표계 X축 기준 병진이동.....	335
2.3.30.	<i>tool.transy</i> : 툴 좌표계 기준 Y축 병진이동.....	338
2.3.31.	<i>tool.transz</i> : 툴 좌표계 기준 Z축 병진이동.....	341
2.3.32.	<i>tool.rot</i> : 툴 좌표계 기준 회전이동.....	344
2.3.33.	<i>tool.rotx</i> : 툴 좌표계 X축 기준 회전이동.....	348
2.3.34.	<i>tool.ropy</i> : 툴 좌표계 기준 Y축 회전이동.....	352
2.3.35.	<i>tool.rotz</i> : 툴 좌표계 기준 Z축 회전이동.....	356
2.3.36.	<i>delay</i> : 대기시간.....	360
2.3.37.	<i>workframe</i> : 워크 좌표계 번호 설정.....	361
2.3.38.	<i>toolframe</i> : 툴 좌표계 번호 설정.....	364
2.4.	힘/순응 제어 스크립트.....	368
2.4.1.	<i>force.enable</i> : 힘 제어 활성화.....	370
2.4.2.	<i>force.disable</i> : 힘 제어 비활성화.....	372
2.4.3.	<i>force.set</i> : 힘 크기 및 방향 설정.....	373
2.4.4.	<i>force.reset</i> : 힘 제어 설정 초기화.....	375
2.4.5.	<i>force.bias</i> : 외력 참조 모드.....	376
2.4.6.	<i>force.set_compliance_x</i> : X 방향 순응 제어 설정.....	377
2.4.7.	<i>force.set_compliance_y</i> : Y 방향 순응 제어 설정.....	379
2.4.8.	<i>force.set_compliance_z</i> : Z 방향 순응 제어 설정.....	381
2.4.9.	<i>force.set_compliance_rx</i> : Rx 방향 순응 제어 설정.....	383
2.4.10.	<i>force.set_compliance_ry</i> : Ry 방향 순응 제어 설정.....	385
2.4.11.	<i>force.set_compliance_rz</i> : Rz 방향 순응 제어 설정.....	387
2.4.12.	<i>force.collision</i> : 충돌 감지 설정.....	389
2.4.13.	<i>force.set_collision</i> : 충돌 감지 감도 설정.....	390
2.4.14.	<i>force.get_collision</i> : 현재 충돌 감지 감도 값 읽기.....	392
2.5.	입/출력 스크립트.....	393
2.5.1.	<i>dio.set</i> : 디지털 출력 신호 활성화(ON).....	397
2.5.2.	<i>dio.reset</i> : 디지털 출력 신호 비활성화(OFF).....	399
2.5.3.	<i>dio.set_bits</i> : 다중 디지털 출력 신호 활성화(ON).....	400
2.5.4.	<i>dio.reset_bits</i> : 다중 디지털 출력 신호 비활성화(OFF).....	402
2.5.5.	<i>dio.get</i> : 디지털 입력 값 가져오기.....	404
2.5.6.	<i>dio.in_wait</i> : 디지털 입력 신호 대기.....	406
2.5.7.	<i>dio.out_wait</i> : 디지털 출력 신호 대기.....	409

2.5.8.	<i>dio.pulse</i> : 디지털 단일 펄스 출력.....	412
2.5.9.	<i>aio.set</i> : 아날로그 출력 신호 설정.....	414
2.5.10.	<i>aio.get</i> : 아날로그 출력 신호 가져오기.....	416
2.5.11.	<i>tool.set</i> : 툴 출력 신호 활성화(ON).....	418
2.5.12.	<i>tool.reset</i> : 툴 출력 신호 비활성화(OFF).....	419
2.5.13.	<i>tool.get</i> : 툴 입력 값 가져오기.....	420
2.5.14.	<i>tool.in_wait</i> : 툴 입력 신호 대기.....	422
2.5.15.	<i>tool.out_wait</i> : 툴 출력 신호 대기.....	424
2.5.16.	<i>tool.voltage</i> : 툴 출력 전압 값 설정.....	426
2.5.17.	<i>tool.modbus_write_register</i> : Holding Register에 값 쓰기.....	427
2.5.18.	<i>tool.modbus_read_register</i> : Holding Register에 값 읽기.....	428
2.6.	다중처리(THREAD) 스크립트.....	430
2.6.1.	<i>thread_name = thread</i> : 스레드 선언.....	431
2.6.2.	<i>thread_name.start</i> : 스레드 시작.....	433
2.6.3.	<i>thread_name.stop</i> : 스레드 정지.....	434
2.6.4.	<i>thread_name.pause</i> : 스레드 일시 정지.....	435
2.6.5.	<i>thread_name.resume</i> : 스레드 재시작.....	436
2.6.6.	<i>thread_name.state</i> : 스레드 상태 확인.....	437
2.6.7.	<i>lock</i> : 스레드 잠금.....	439
2.6.8.	<i>unlock</i> : 스레드 잠금 해제.....	441
2.7.	사용자 메시지 출력 스크립트.....	442
2.7.2.	<i>msg.error</i> : 에러 메시지.....	443
2.7.3.	<i>msg.warn</i> : 경고 메시지.....	445
2.7.4.	<i>msg.info</i> : 정보 메시지.....	447
2.8.	수학 함수 스크립트.....	449
2.8.1.	<i>math.pi</i> : 파이.....	450
2.8.2.	<i>math.deg</i> : Degree 각도.....	451
2.8.3.	<i>math.rad</i> : Radian 각도.....	453
2.8.4.	<i>math.sqrt</i> : 제곱근.....	455
2.8.5.	<i>math.pow</i> : 거듭제곱.....	457
2.8.6.	<i>math.abs</i> : 절대 값.....	459
2.8.7.	<i>math.ceil</i> : 올림.....	461
2.8.8.	<i>math.floor</i> : 내림.....	463
2.8.9.	<i>math.sin</i> : 사인함수.....	465
2.8.10.	<i>math.cos</i> : 코사인 함수.....	467
2.8.11.	<i>math.tan</i> : 탄젠트 함수.....	469
2.8.12.	<i>math.asin</i> : 아크사인 함수.....	471
2.8.13.	<i>math.acos</i> : 아크코사인 함수.....	473

2.8.14.	<i>math.atan</i> : 아크탄젠트 함수	475
2.8.15.	<i>math.atan2</i> : 제2아크탄젠트 함수	477
2.9.	TCP/IP 통신 서버 스크립트	479
2.9.1.	<i>serv = server</i> : 서버 선언	480
2.9.2.	<i>serv.open</i> : 서버 열기	481
2.9.3.	<i>serv.close</i> : 서버 종료	483
2.9.4.	<i>serv.state</i> : 클라이언트 접속 상태 확인	484
2.9.5.	<i>serv.flush</i> : 송/수신 버퍼 초기화	486
2.9.6.	<i>serv.read</i> : 데이터 수신	487
2.9.7.	<i>serv.write</i> : 데이터 송신	489
2.9.8.	<i>serv.set_label</i> : 송신 데이터 접미사 설정	491
2.10.	TCP/IP 통신 클라이언트 스크립트	493
2.10.1.	<i>cli = client</i> : 서버 선언	495
2.10.2.	<i>cli.open</i> : 서버 연결	497
2.10.3.	<i>cli.close</i> : 서버 연결 종료	499
2.10.4.	<i>cli.state</i> : 서버와 연결 상태 확인	500
2.10.5.	<i>cli.flush</i> : 송/수신 버퍼 초기화	502
2.10.6.	<i>cli.write</i> : 문자열 데이터 송신	503
2.10.7.	<i>cli.set_label</i> : 송신 문자열 데이터 접미사 설정	505
2.10.8.	<i>cli.send</i> : 버퍼에 쌓인 데이터 송신	507
2.10.9.	<i>cli.add_byte</i> : send 버퍼에 부호없는 8비트 정수형(byte) 데이터를 추가	509
2.10.10.	<i>cli.add_short</i> : send 버퍼에 16비트 정수형(short) 데이터를 추가	511
2.10.11.	<i>cli.add_ushort</i> : send 버퍼에 부호없는 16비트 정수형(unsigned short) 데이터를 추가 513	
2.10.12.	<i>cli.add_int</i> : send 버퍼에 32비트 정수형(integer) 데이터를 추가	515
2.10.13.	<i>cli.add_uint</i> : send 버퍼에 부호없는 32비트 정수형(unsigned integer) 데이터를 추 가 517	
2.10.14.	<i>cli.add_float</i> : send 버퍼에 32비트 실수형(float) 데이터를 추가	519
2.10.15.	<i>cli.add_double</i> : send 버퍼에 64비트 실수형(double) 데이터를 추가	521
2.10.16.	<i>cli.read</i> : 문자열 데이터 수신	523
2.10.17.	<i>cli.recv</i> : 서버로부터 수신버퍼에 데이터를 수신	525
2.10.18.	<i>cli.get_byte</i> : recv 버퍼로부터 부호없는 8비트 정수형(byte) 데이터 읽기	527
2.10.19.	<i>cli.get_short</i> : recv 버퍼로부터 16비트 정수형(short) 데이터 읽기	529
2.10.20.	<i>cli.get_ushort</i> : recv 버퍼로부터 부호없는 16비트 정수형(unsigned short) 데이터 읽기 531	
2.10.21.	<i>cli.get_int</i> : recv 버퍼로부터 32비트 정수형(integer) 데이터 읽기	533
2.10.22.	<i>cli.get_uint</i> : recv 버퍼로부터 부호없는 32비트 정수형(unsigned integer) 데이터 읽기 535	

2.10.23.	<i>cli.get_float : recv 버퍼로부터 32비트 실수형(float) 데이터 읽기</i>	537
2.10.24.	<i>cli.get_double : recv 버퍼로부터 64비트 실수형(double) 데이터 읽기</i>	539
2.11.	MODBUS 통신 서버 스크립트	541
2.11.1.	<i>modserv = modbus_server : Modbus 서버 선언</i>	542
2.11.2.	<i>modserv.read_bit : Coil 값 읽기</i>	543
2.11.3.	<i>modserv.read_bits : 여러 개의 Coil 값 읽기</i>	546
2.11.4.	<i>modserv.read_bit_multi : 해당 이름의 모든 Coil 값 읽기</i>	549
2.11.5.	<i>modserv.read_register : Holding Register 값 읽기</i>	552
2.11.6.	<i>modserv.read_registers : 여러 개의 Holding Register 값 읽기</i>	555
2.11.7.	<i>modserv.read_register_multi : 해당 이름의 모든 Holding Register 값 읽기</i>	558
2.11.8.	<i>modserv.write_bit : Coil 값 쓰기</i>	561
2.11.9.	<i>modserv.write_bits : 여러 개의 Coil 값 쓰기</i>	564
2.11.10.	<i>modserv.write_bit_multi : 해당 이름의 모든 Coil에 값 쓰기</i>	567
2.11.11.	<i>modserv.write_register : Holding Register 값 쓰기</i>	570
2.11.12.	<i>modserv.write_registers : 여러 개의 Holding Register 값 쓰기</i>	573
2.11.13.	<i>modserv.write_register_multi : 해당 이름의 모든 Holding Register에 값 쓰기</i>	576
2.12.	MODBUS 통신 클라이언트 스크립트	579
2.12.1.	<i>modcli = modbus_client() : Modbus 클라이언트 선언</i>	580
2.12.2.	<i>modcli.open : Modbus 서버와 연결</i>	581
2.12.3.	<i>modcli.close : Modbus 서버와 연결 해제</i>	583
2.12.4.	<i>modcli.read_bit : Coil 값 읽기</i>	585
2.12.5.	<i>modcli.read_bits : 여러 개의 Coil 값 읽기</i>	587
2.12.6.	<i>modcli.read_register : Holding Register 값 읽기</i>	589
2.12.7.	<i>modcli.read_registers : 여러 개의 Holding Register 값 읽기</i>	591
2.12.8.	<i>modcli.write_bit : Coil에 값 쓰기</i>	593
2.12.9.	<i>modcli.write_bits : 여러 개의 Coil에 값 쓰기</i>	595
2.12.10.	<i>modcli.write_register : Holding Register에 값 쓰기</i>	597
2.12.11.	<i>modcli.write_registers : 여러 개의 Holding Register에 값 쓰기</i>	599
2.13.	시리얼 통신 스크립트	601
2.13.1.	<i>seri = serial : 시리얼 포트 선언</i>	603
2.13.2.	<i>seri.open : 시리얼 통신 연결</i>	605
2.13.3.	<i>seri.close : 시리얼 통신 종료</i>	607
2.13.4.	<i>seri.state : 시리얼 통신 포트 상태 확인</i>	609
2.13.5.	<i>seri.read : 데이터 수신</i>	611
2.13.6.	<i>seri.write : 데이터 송신</i>	613
2.13.7.	<i>seri.set_byte_timeout : 수신 데이터 문자간 타임아웃 설정</i>	615
2.13.8.	<i>seri.set_label : 송신 데이터 접미사 설정</i>	617
2.13.9.	<i>seri.count_rx : 수신 버퍼 데이터 개수 반환</i>	620

2.13.10.	<i>seri.count_tx</i> : 송신 버퍼 데이터 개수 반환	622
2.13.11.	<i>seri.flush_rx</i> : 수신 버퍼 초기화.....	624
2.13.12.	<i>seri.flush_tx</i> : 송신 버퍼 초기화.....	626
2.13.13.	<i>seri.flush</i> : 송/수신 버퍼 초기화.....	628
2.14.	전역변수 스크립트	630
2.14.1.	<i>var.p</i> : 위치 값 읽기.....	631
2.14.2.	<i>var.p</i> : 위치 값 쓰기.....	635
2.14.3.	<i>var.i</i> : 정수 값 읽기	637
2.14.4.	<i>var.i</i> : 정수 값 쓰기	639
2.14.5.	<i>var.f</i> : 실수 값 읽기.....	641
2.14.6.	<i>var.f</i> : 실수 값 쓰기.....	643
2.14.7.	<i>var.b</i> : 논리형 읽기	645
2.14.8.	<i>var.b</i> : 논리형 쓰기	647
2.14.9.	<i>var.s</i> : 문자열 읽기.....	649
2.14.10.	<i>var.s</i> : 문자열 쓰기.....	651
2.15.	서브루틴(SUBROUTINE) 스크립트.....	653
2.15.1.	<i>call</i> : 서브루틴 호출.....	654
2.16.	타이머 스크립트	656
2.16.1.	<i>timer = time</i> : 타이머 생성.....	657
2.16.2.	<i>timer.start</i> : 타이머 시작.....	658
2.16.3.	<i>timer.stop</i> : 타이머 정지.....	659
2.16.4.	<i>timer.elapsed</i> : 소요시간 계산.....	660
2.17.	프로그램 제어 스크립트	662
2.17.1.	<i>robot.pause</i> : 프로그램 일시정지.....	663
2.17.2.	<i>robot.stop</i> : 프로그램 정지.....	664

머리말

(주)푸른기술 제품을 구매해 주셔서 감사합니다. 제품을 설치하기 전에 본 사용자메뉴얼을 모두 읽고 설치과정에 따라 설명서의 지시 사항을 따라야 합니다. 본 메뉴얼의 내용은 메뉴얼이 작성된 때를 기준으로 하며, 제품에 대한 정보는 사용자에게 알리지 않고 변경될 수 있습니다.

본 메뉴얼에서 기술하고 있는 요구 조건이나 권장 사항 또는 안전 절차에 대해 확실하지 않은 부분은 (주)푸른기술 또는 (주)푸른로보틱스에 문의하시길 바랍니다. 본 메뉴얼의 일부 그림은 시스템의 개념 및 설치에 대한 이해를 돕기 위한 것이며, 실제 제품과 다를 수 있습니다.

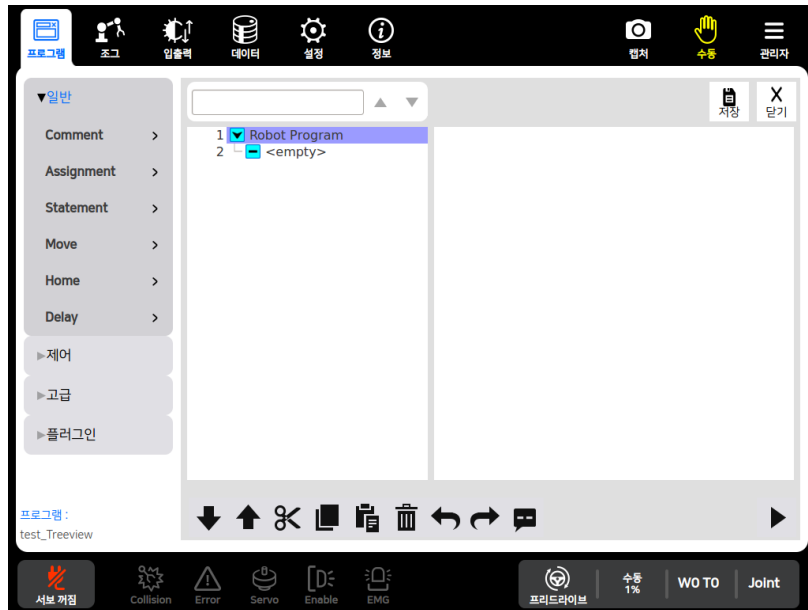
본 메뉴얼의 모든 내용과 도안에 대한 저작권, 지적재산권은 (주)푸른기술에 있습니다. 따라서 (주)푸른기술의 사전 서면 허락없이 사용, 복사, 유포, 배포하는 어떠한 행위도 금지되며, (주)푸른기술의 지적재산권 침해에 해당됩니다.

본 장비의 특허권을 오용하거나 변경 적용하는데 책임은 전적으로 사용자에게 있으며, 본 메뉴얼에서 다루는 정보는 신뢰할 수 있는 정보지만 오류 또는 오탈자로 인한 손실에 대해 어떠한 책임도 지지 않습니다. 제품 개선에 따라 메뉴얼에 포함된 정보는 예고없이 변경될 수 있습니다.

본 메뉴얼에서 제공되는 정보는 (주)푸른기술의 자산이며, 동의없이 전부 또는 일부를 복제할 수 없습니다. 메뉴얼 개정에 관한 상세한 정보는 인터넷 홈페이지(www.puloon.co.kr)를 방문하기 바랍니다.

© 2023 Puloon Technology Inc. All rights reserved

1 트리뷰 프로그램



- 트리뷰 프로그램은 왼쪽에 있는 명령어 노드와 가운데에 있는 트리뷰, 오른쪽에 선택한 명령어와 관련된 설정 정보들이 표시됩니다.
- 명령어는 크게 일반, 제어, 고급, 플러그인으로 구성되어 있으며 원하는 명령어를 선택하여 오른쪽에 팝업되는 설정 화면에 따라 설정값을 입력합니다.

항목	설명
일반	기본적인 명령어들로 구성되어 있습니다. 변수 선언, 이동, 지연 등 기본 명령어를 포함한 Statement 명령어에 입출력, 모드버스 및 TCP/IP 통신, 힘 제어 등 다양한 명령어들로 구성되어 있습니다.
제어	반복문과 조건문, Break 와 Return 문 등으로 구성되어 있습니다.
고급	함수 생성과 다른 프로그램을 호출할 수 있는 명령어로 구성되어 있습니다.
플러그인	메인 메뉴 '설정'의 플러그인 메뉴에서 생성한 플러그인 인스턴스들로 구성되어 있습니다.

- 명령어 설정 방법은 아래의 글들을 참고하시고 트리뷰 프로그램의 기본 버튼 조작 방법은 사용자 매뉴얼을 참고 바랍니다.

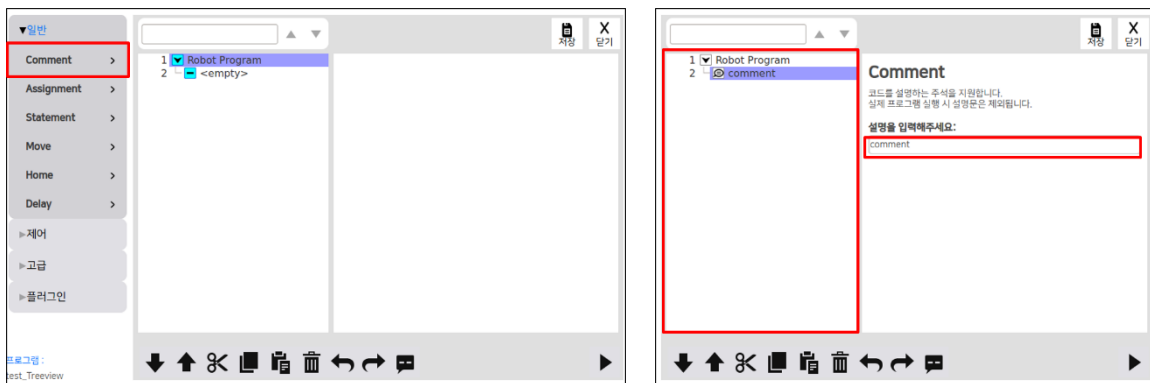
1.1. 기본 명령어 설정

1.1.1 일반

기본적인 명령어들로 구성되어 있습니다. 변수 선언, 이동, 지연 등 기본 명령어를 포함한 Statement 명령어에 입출력, 모드버스 및 TCP/IP 통신, 힘 제어 등 다양한 명령어들로 구성되어 있습니다.

1.1.1.1. Comment

- 코드를 설명하는 주석 명령어 입니다. 트리뷰 어느 라인에서든 추가가 가능합니다.
- 주석 설명이므로 프로그램 실행에 영향을 끼치지 않습니다.



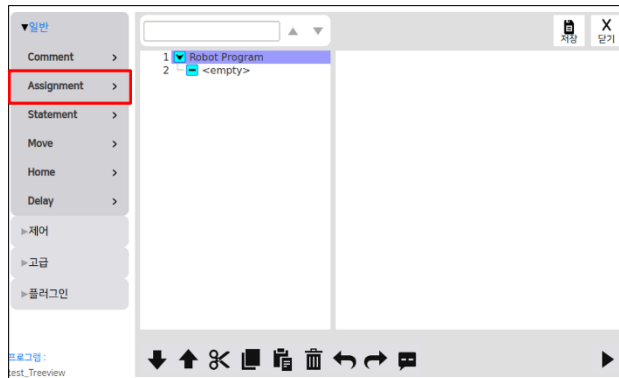
- 좌측에 있는 일반 그룹의 Comment 트리 메뉴를 클릭합니다.
- 트리뷰 편집 화면에 'Comment' 내용의 Comment 명령어가 추가되고 명령어 설정 화면에 해당 명령어의 설정 화면이 나타납니다. 설명을 입력하는 텍스트 창을 클릭합니다.



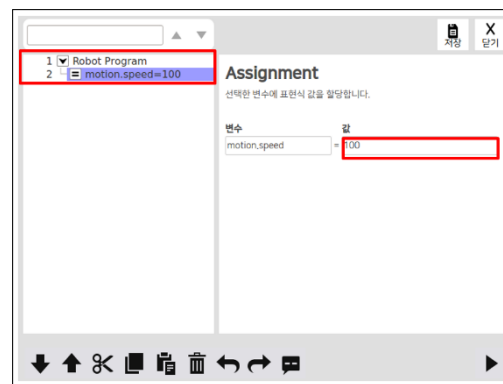
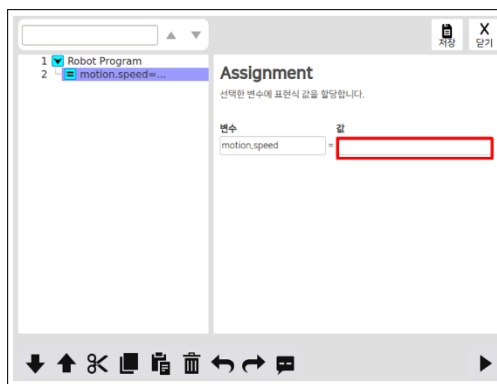
- 키보드가 팝업됩니다. 주석 설명문을 입력한 후 'Ent' 버튼을 클릭합니다.
- 트리뷰 편집 화면에 입력한 설명문으로 변경된 것을 확인할 수 있습니다.

1.1.1.2. Assignment

- 변수를 선언하는 명령어 입니다. 변수명과 그 변수에 맞는 표현식 값을 설정합니다.



- 먼저 변수 칸을 클릭하면 키보드가 팝업됩니다.



- 변수를 입력한 후 값 칸을 클릭하여 키보드가 팝업되면 값을 입력합니다.
- 변수와 값이 모두 입력이 되면 트리뷰 편집 화면에서 하늘색 노드에서 흰색 노드로 색 변경이 되며 입력이 잘 된 것을 확인할 수 있습니다.

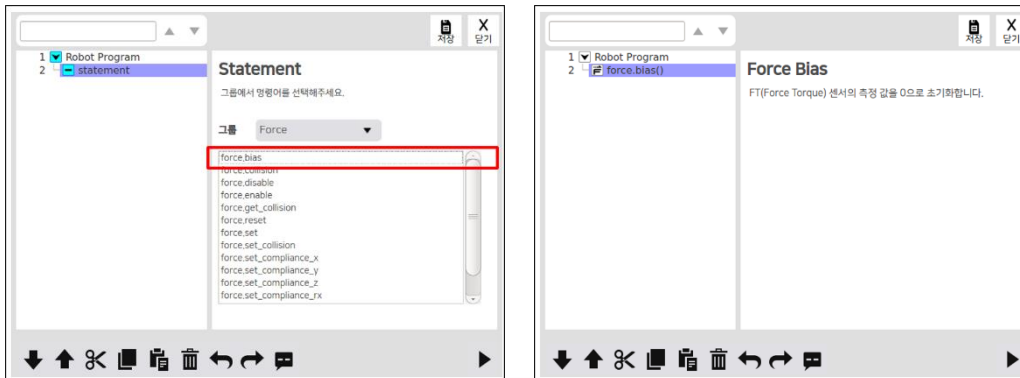
1.1.1.3. Statement

- 힘 제어, I/O, 모드 버스 통신, TCP/IP 통신, 시리얼 통신, 메시지 팝업, 전역 변수, 수학 관련 명령어들을 선택하여 설정합니다.

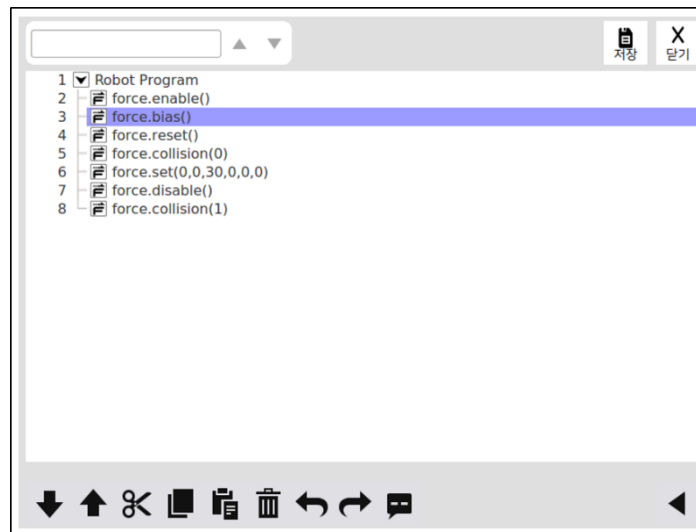
1.1.1.3.1 Force

1. Bias

- force.bias 명령어는 FT 센서 측정값을 0 으로 초기화하는 명령어 입니다.



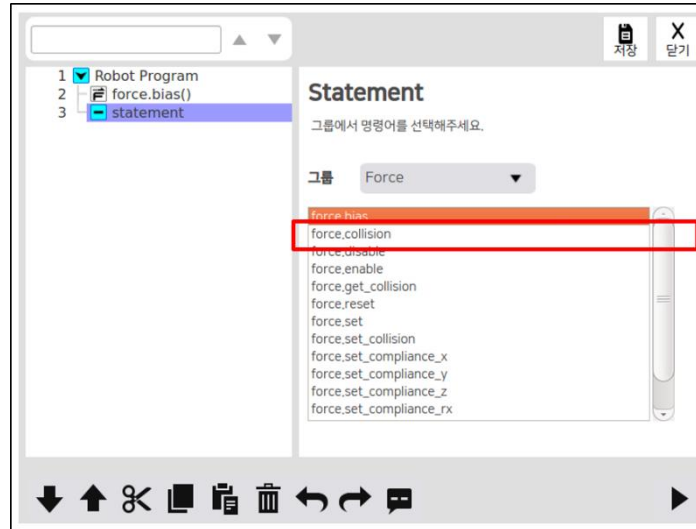
- Statement 의 Force 그룹에 있는 bias 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



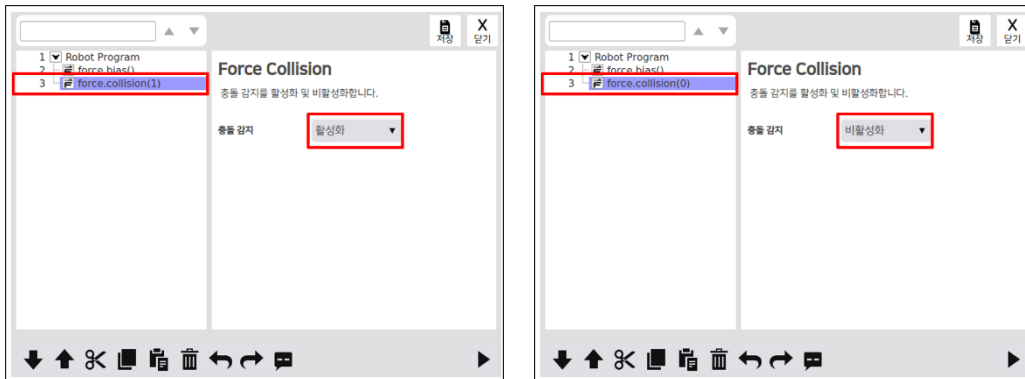
- bias 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

2. Collision

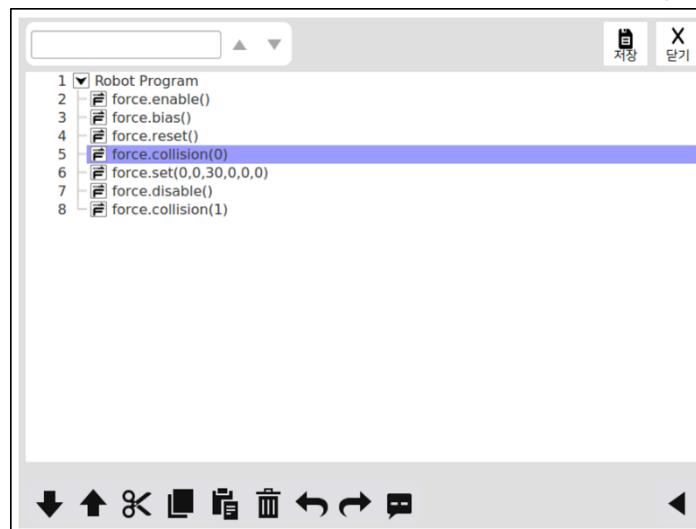
- force.collison 명령어는 충돌 감지 기능을 활성화 및 비활성화 하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 collision 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



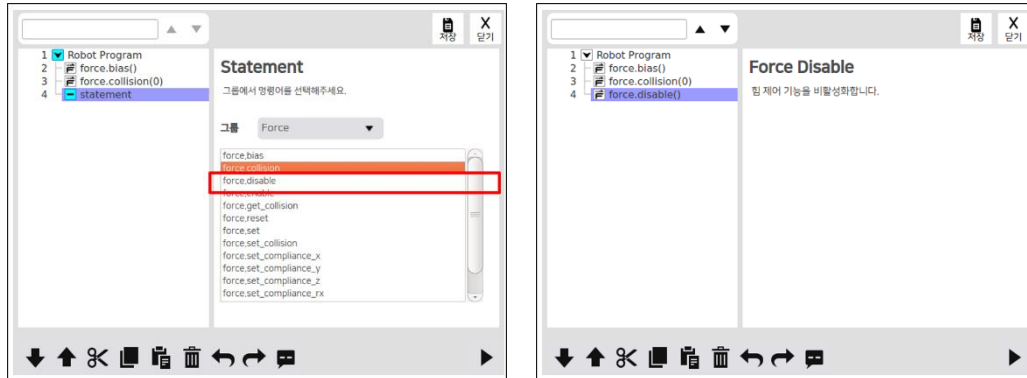
- 명령어 설정 화면에서 콤보 박스를 통해 활성화/비활성화를 선택합니다. (활성화=1, 비활성화=0)



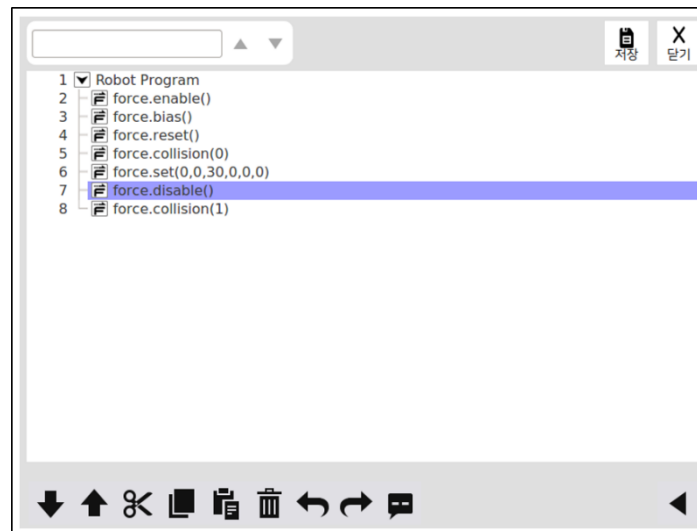
- collision 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

3. Disable

- `force.disable` 은 힘 제어 기능을 비활성화 하는 명령어 입니다.



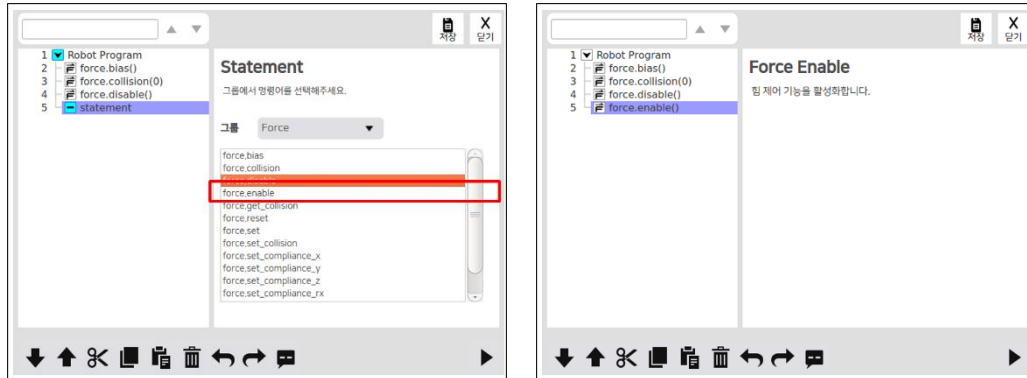
- Statement 의 Force 그룹에 있는 disable 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



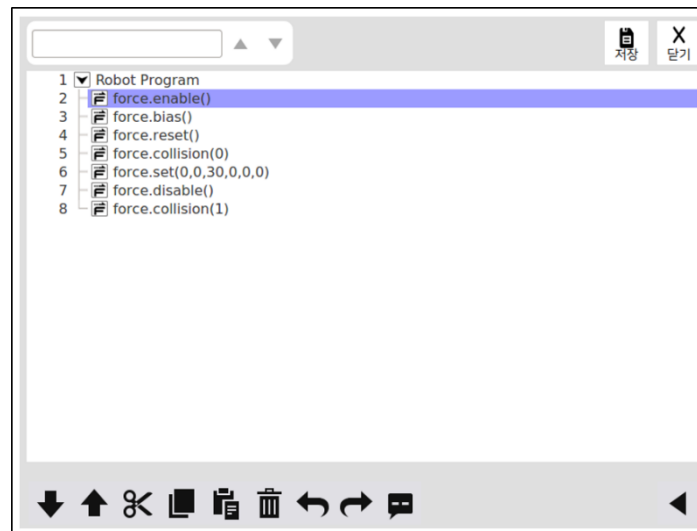
- disable 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

4. Enable

- force.enable 은 힘 제어 기능을 활성화 하는 명령어 입니다.



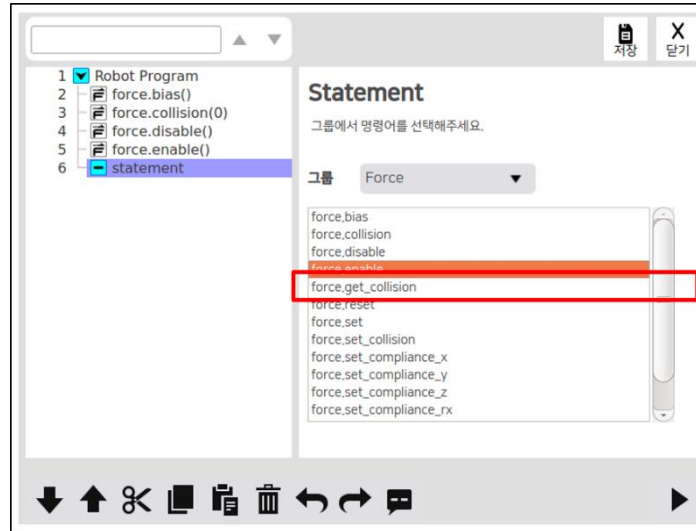
- Statement 의 Force 그룹에 있는 enable 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



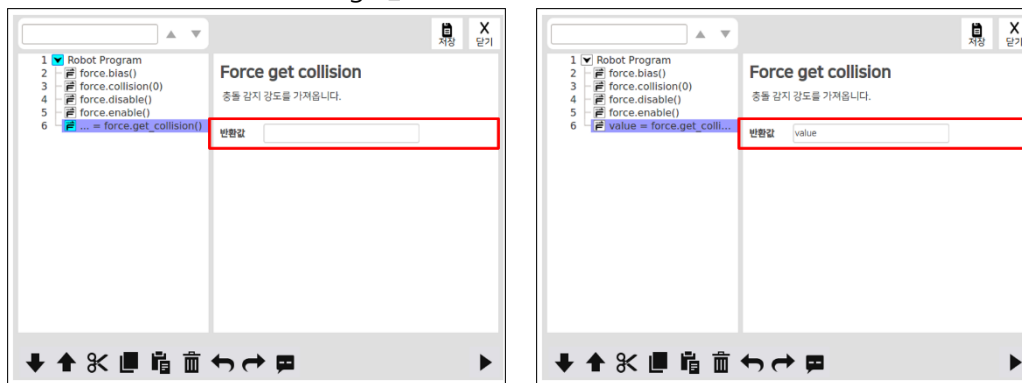
- enable 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

5. Get_collision

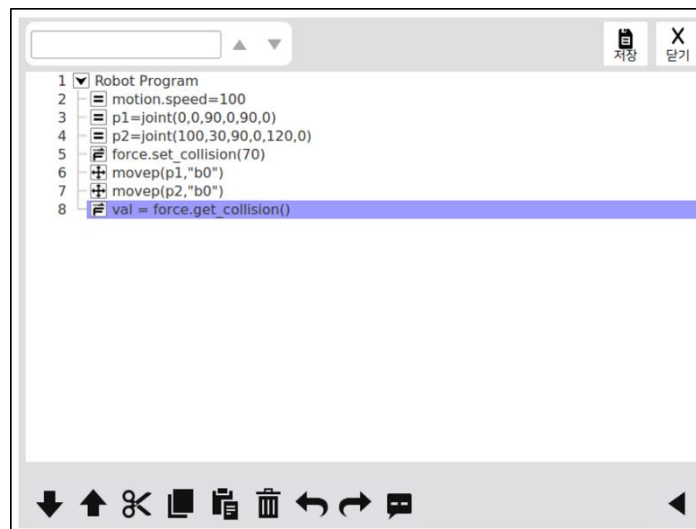
- force.get_collision 은 현재 설정된 충돌 감지의 감도값을 읽어오는 명령어 입니다.



- Statement 의 Force 그룹에 있는 get_collision 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



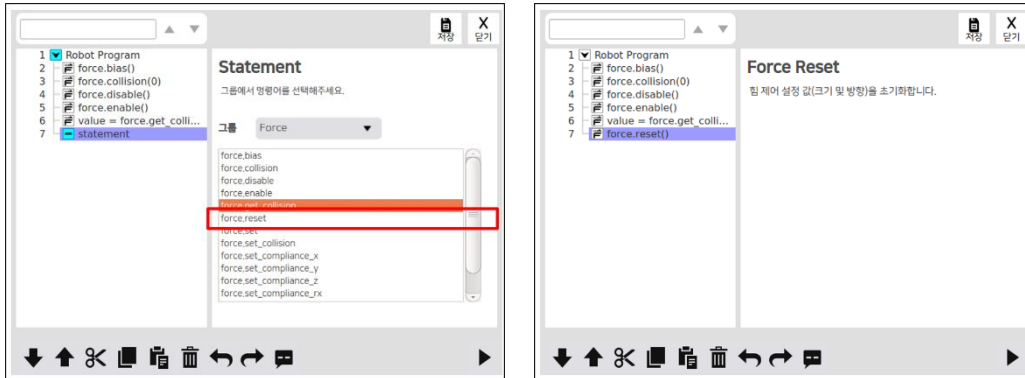
- 명령어 설정 화면에서 키보드를 통해 반환값을 입력합니다.



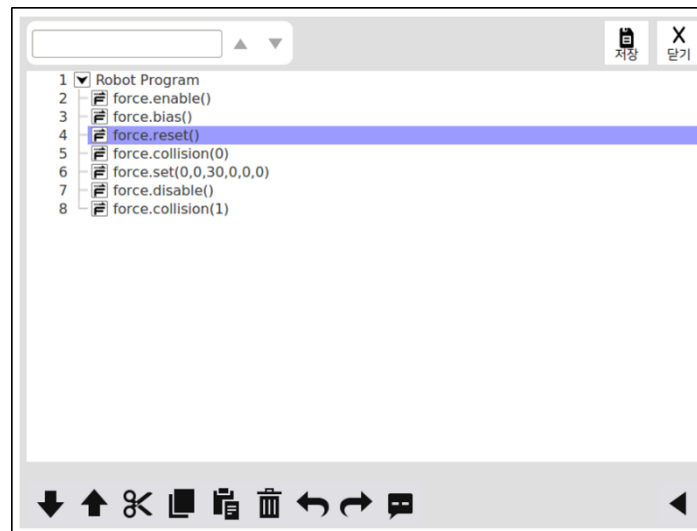
- get_collision 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

6. Reset

- `force.reset` 은 힘 제어 설정 값(크기 및 방향)을 초기화 하는 명령어 입니다.



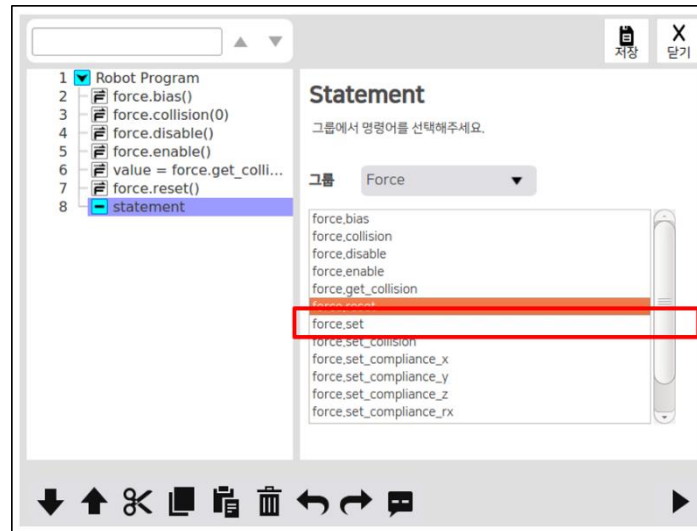
- Statement 의 Force 그룹에 있는 reset 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



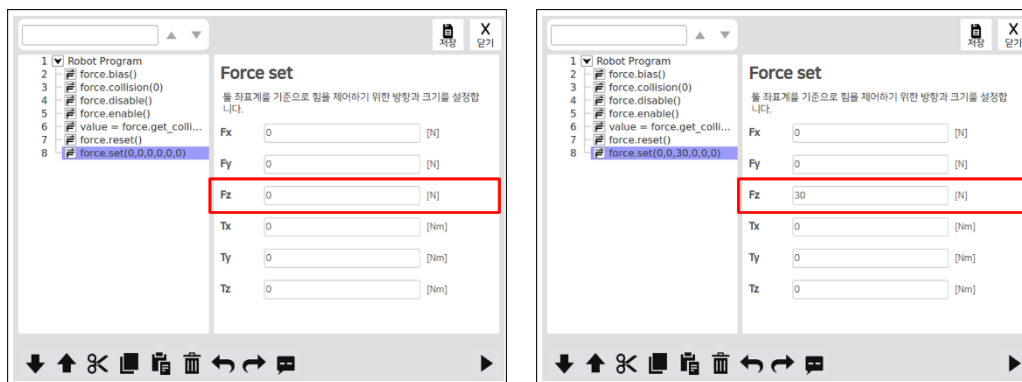
- reset 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

7. Set

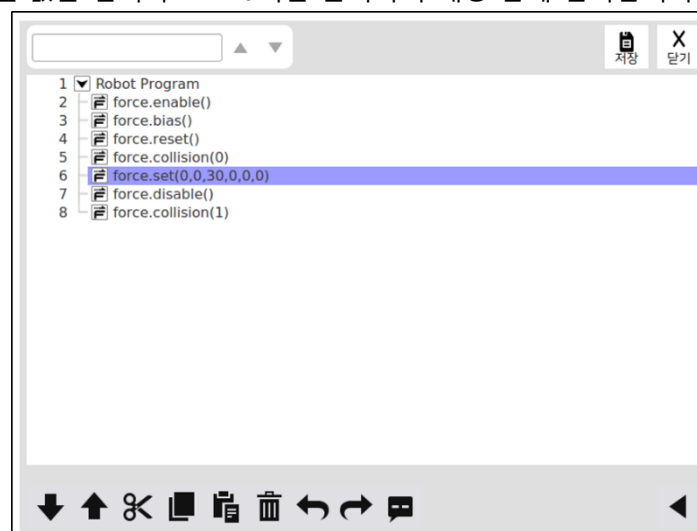
- force.set 은 톨 좌표계를 기준으로 힘을 제어하기 위한 방향과 크기를 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



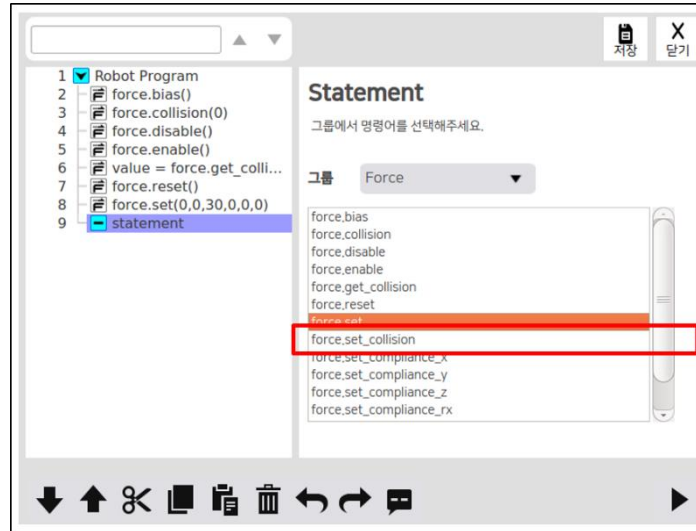
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



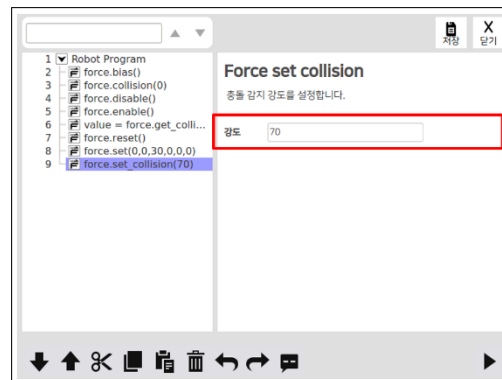
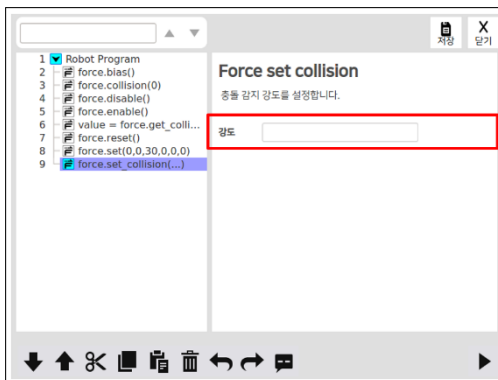
- set 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

8. Set_collision

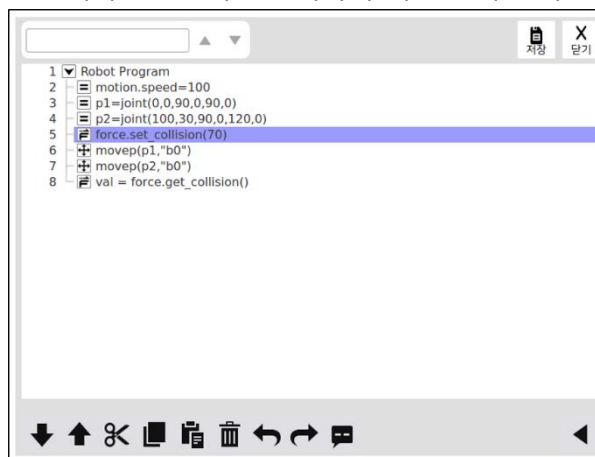
- force.set_collision 은 충돌감지의 감도를 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_collision 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



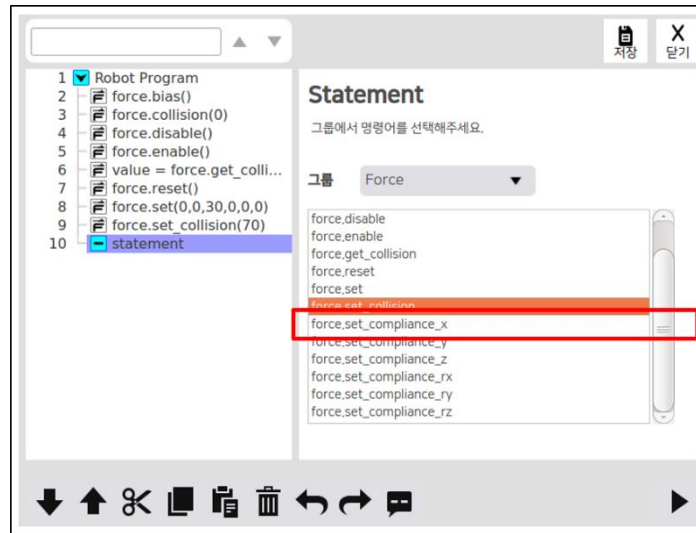
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



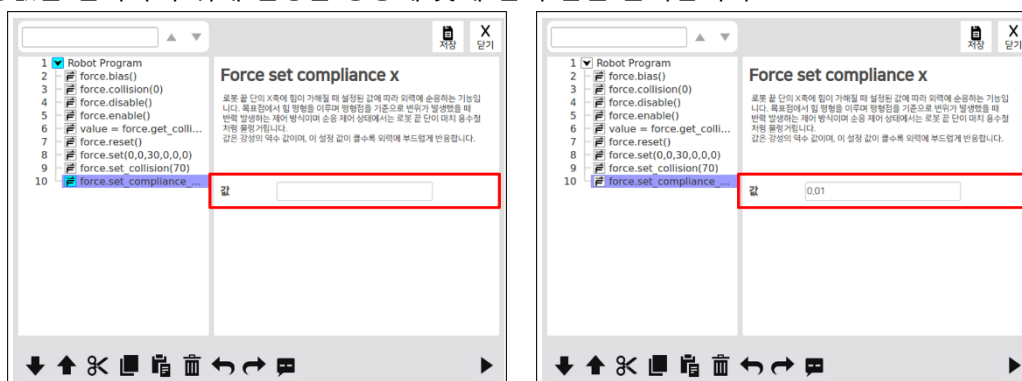
- set_collision 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

9. Set_compliance_x

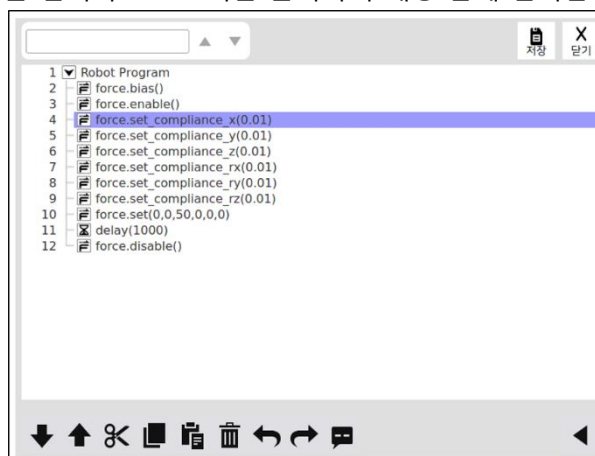
- force.set_compliance_x 는 X 방향의 순응 제어 값을 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_compliance_x 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



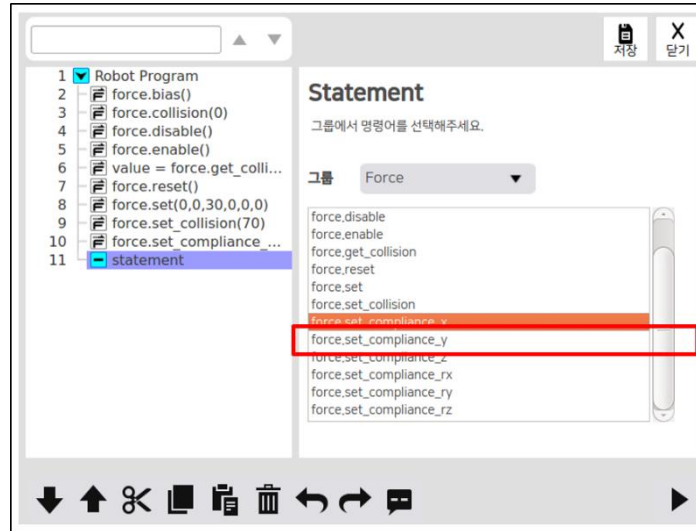
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



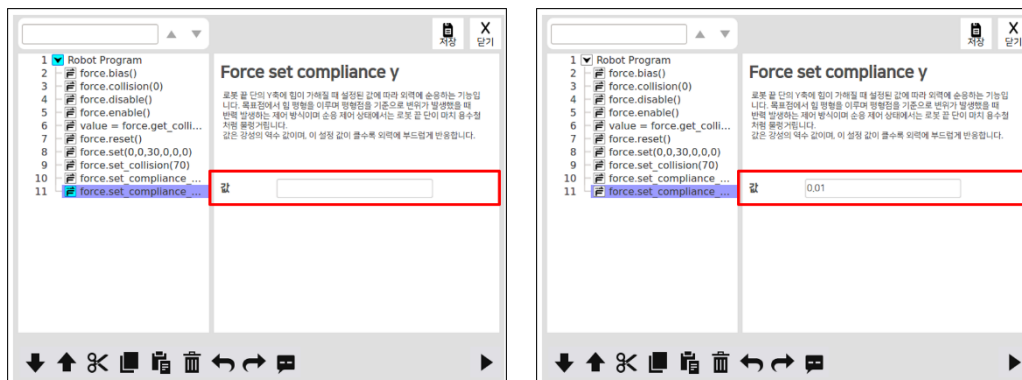
- set_compliance_x 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

10. Set_compliance_y

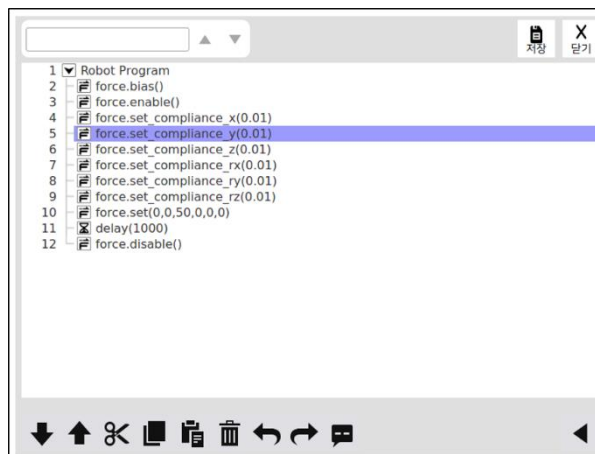
- force.set_compliance_y 는 Y 방향의 순응 제어 값을 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_compliance_y 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



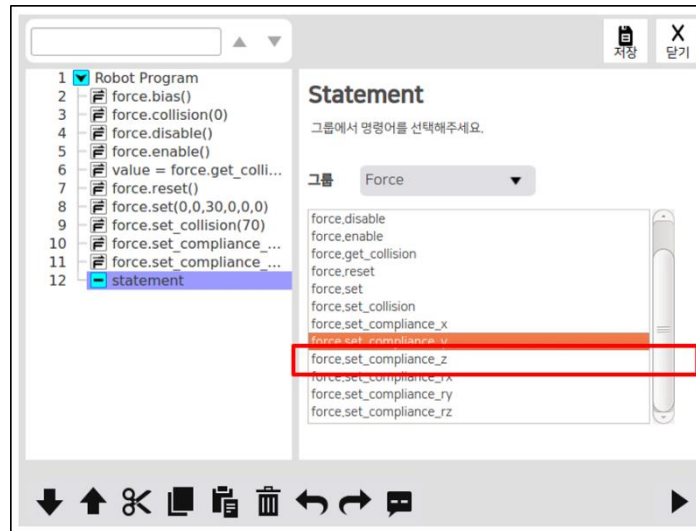
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



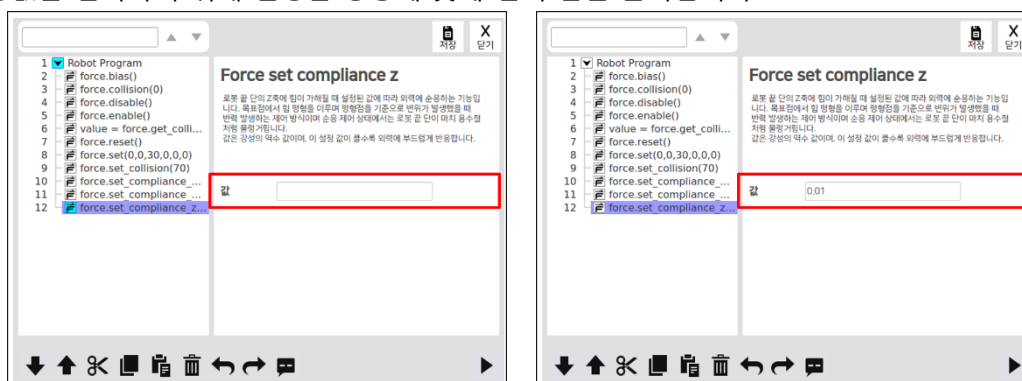
- set_compliance_y 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

11. Set_compliance_z

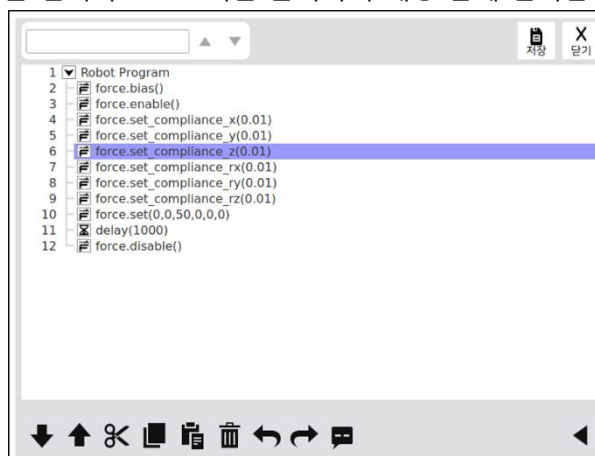
- force.set_compliance_z 는 Z 방향의 순응 제어 값을 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_compliance_z 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



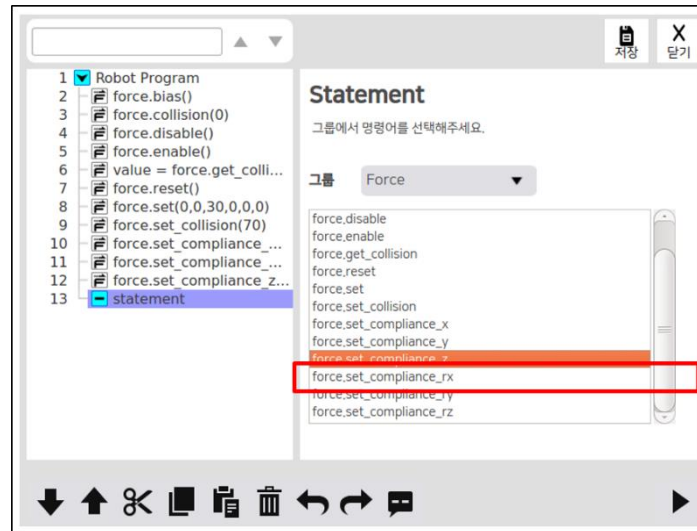
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



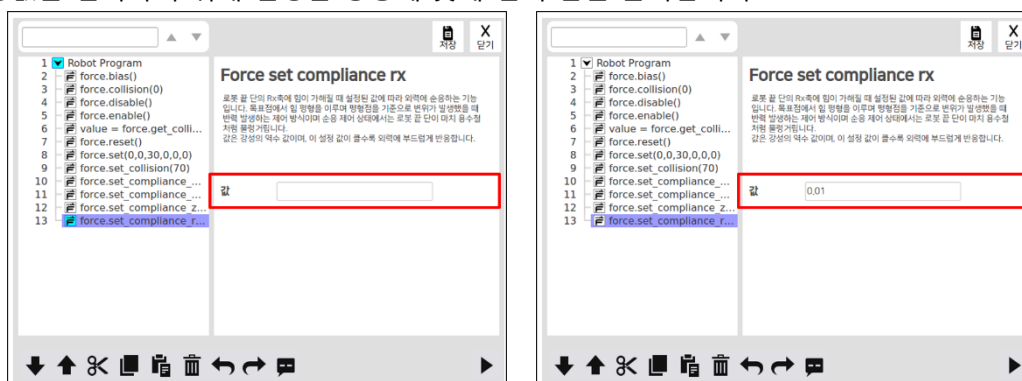
- set_compliance_z 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

12. Set_compliance_rx

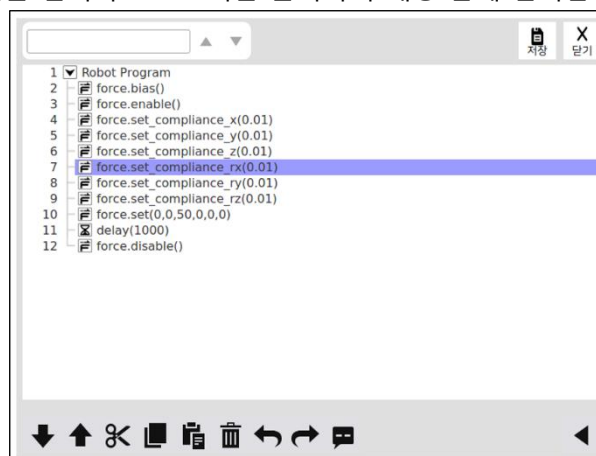
- force.set_compliance_rx 는 Rx 방향의 순응 제어 값을 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_compliance_rx 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



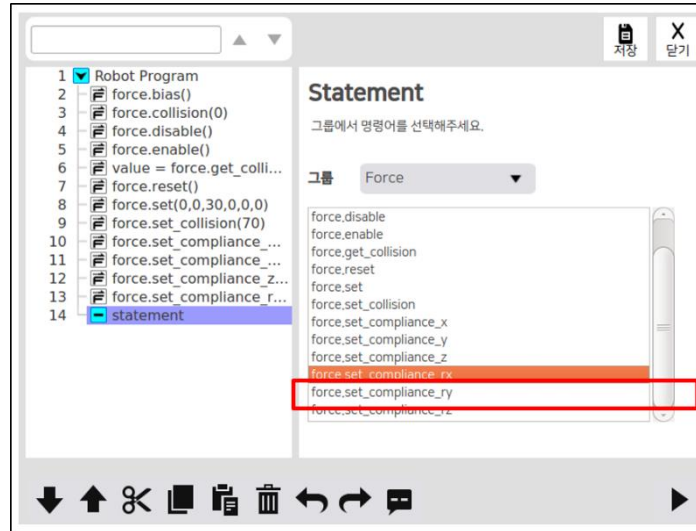
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



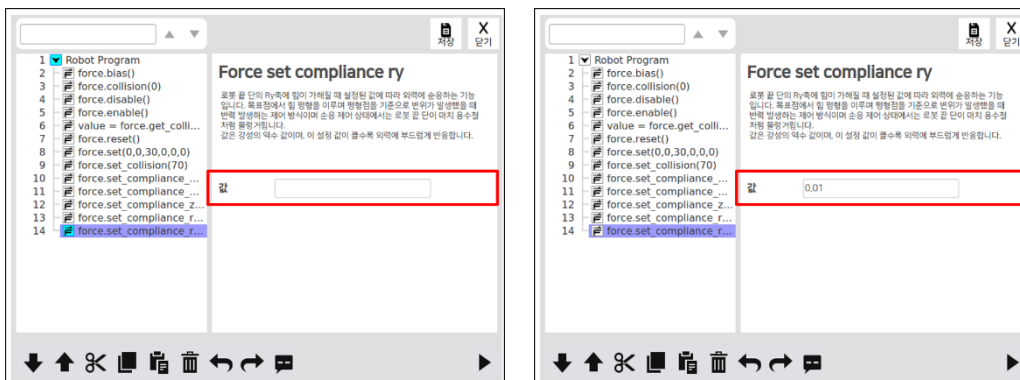
- set_compliance_rx 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

13. Set_compliance_ry

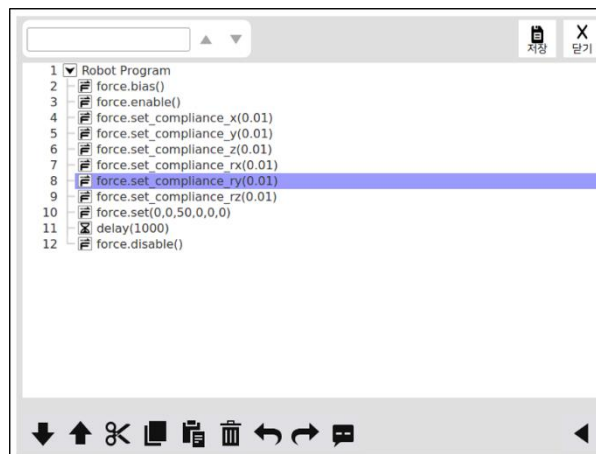
- force.set_compliance_ry 는 Ry 방향의 순응 제어 값을 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_compliance_ry 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



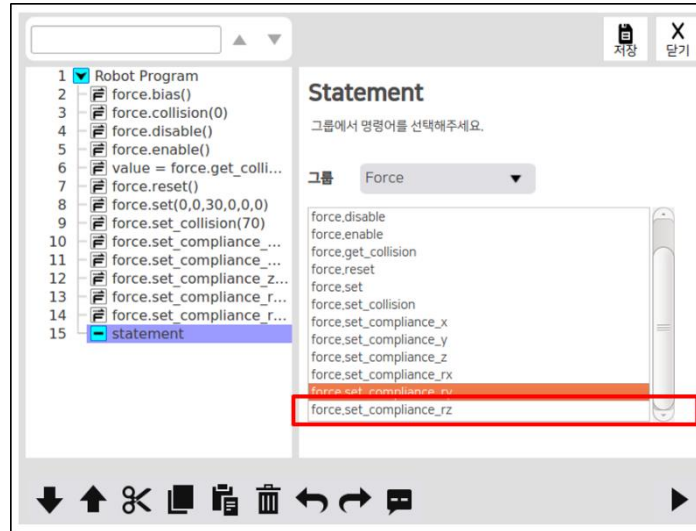
- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.



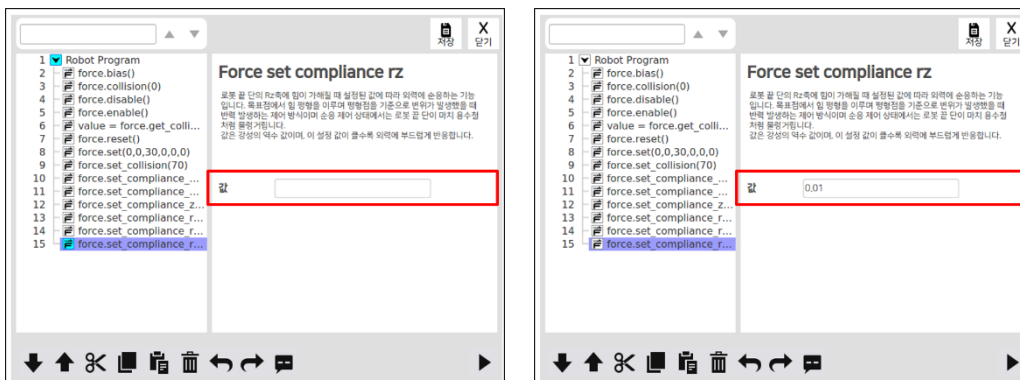
- set_compliance_ry 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

14. Set_compliance_rz

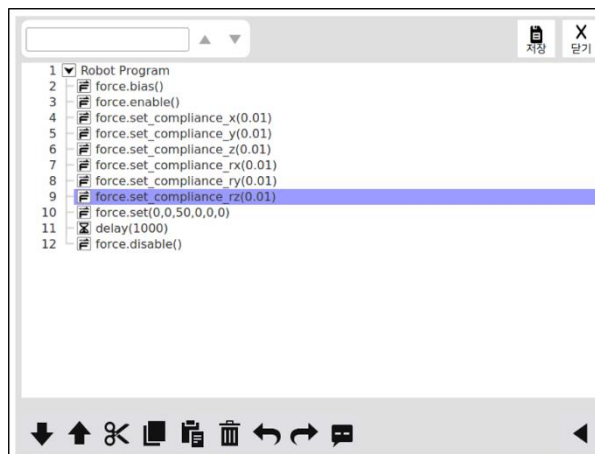
- force.set_compliance_rz 는 Rz 방향의 순응 제어 값을 설정하는 명령어 입니다.



- Statement 의 Force 그룹에 있는 set_compliance_rz 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 설정값을 입력하기 위해 설정할 방향에 맞게 입력 칸을 클릭합니다.



- 키보드가 팝업되면 값을 입력하고 'Ent'키를 클릭하여 해당 칸에 입력합니다.

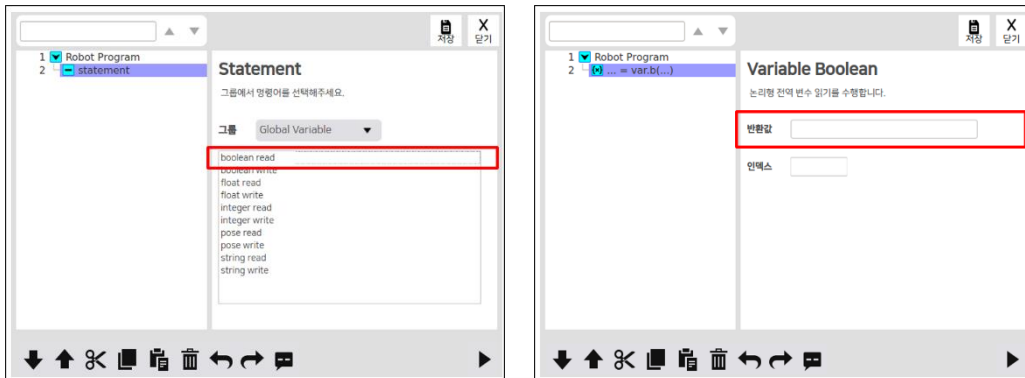


- set_compliance_rz 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

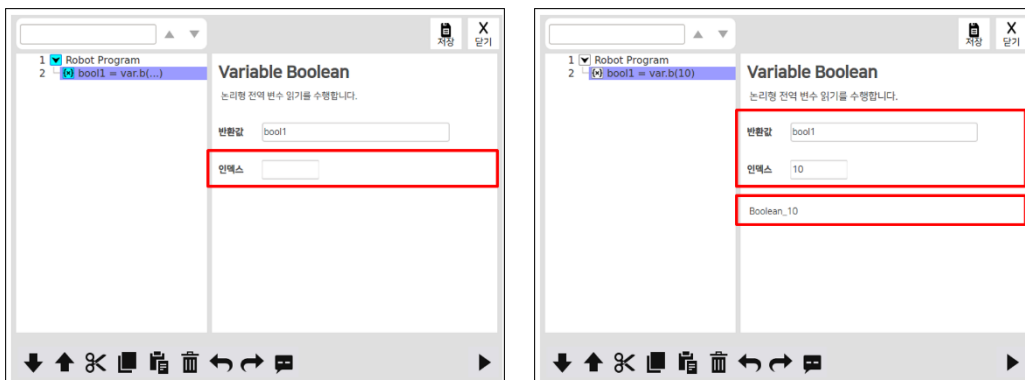
1.1.1.3.2 Global Variable

1. Boolean read

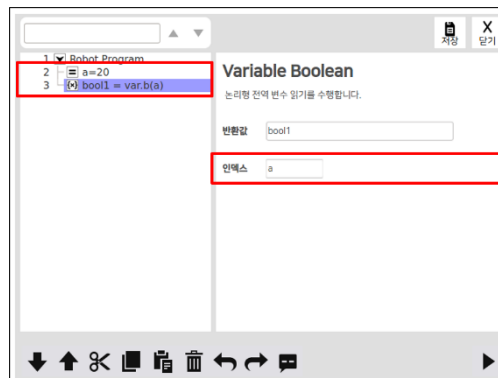
- var.b() 읽기 명령어는 데이터 메뉴에 있는 논리형 전역 변수를 읽어오는 명령어 입니다.



- Statement 의 Global Variable 그룹에 있는 boolean read 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 읽어온 논리형 변수 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

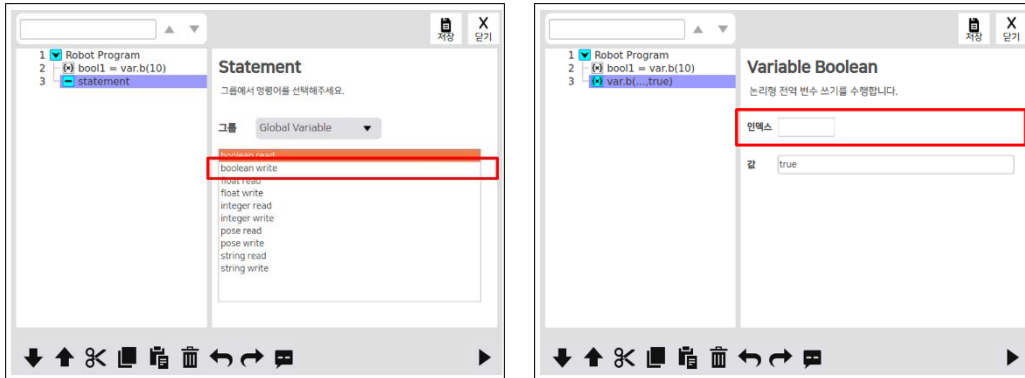


- 1~2000 까지의 논리형 변수 중에 읽어올 인덱스 값을 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.
- 1~2000 까지의 숫자 중에 변수로 선언되어 있는 경우 (ex. a=20), 인덱스에 변수도 입력 가능합니다.

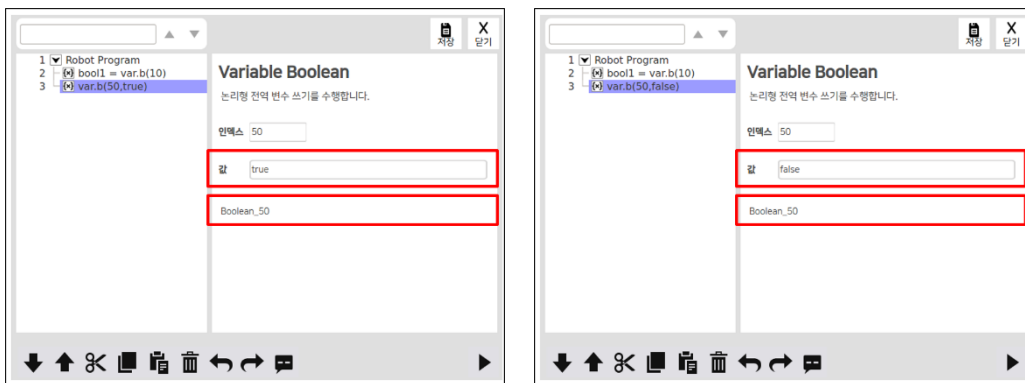


2. Boolean write

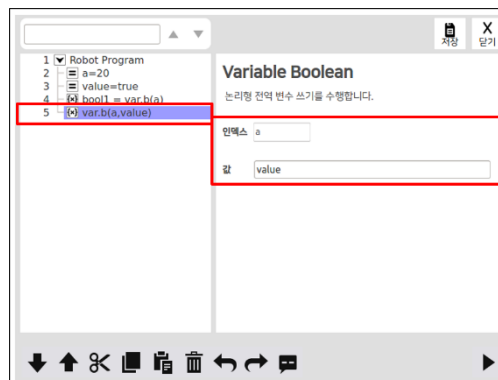
- var.b() 쓰기 명령어는 데이터 메뉴에 있는 논리형 전역 변수를 쓰는 명령어입니다.



- Statement의 Global Variable 그룹에 있는 boolean write 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 1~2000까지의 논리형 변수 중에 읽어올 인덱스 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.

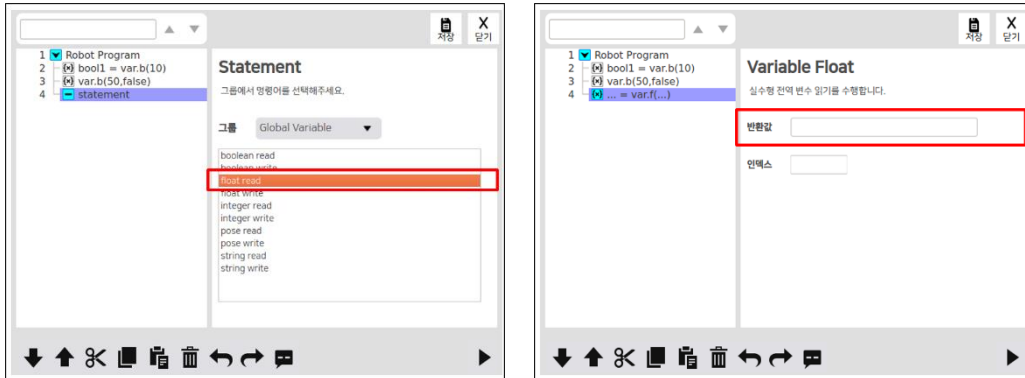


- 값에 논리형 값 중 true/false를 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다.
- 인덱스와 값 모두 선언되어 있는 변수로 입력 가능합니다.

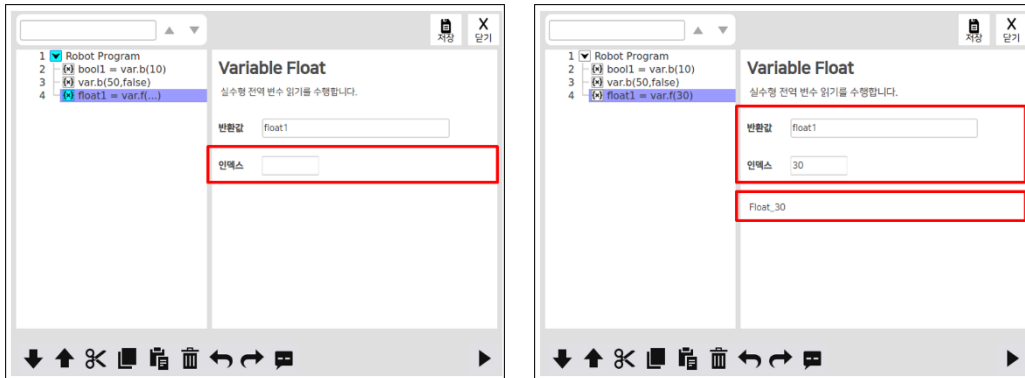


3. Float read

- var.f() 읽기 명령어는 데이터 메뉴에 있는 실수형 전역 변수를 읽어오는 명령어 입니다.



- Statement 의 Global Variable 그룹에 있는 float read 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 읽어온 실수형 변수 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

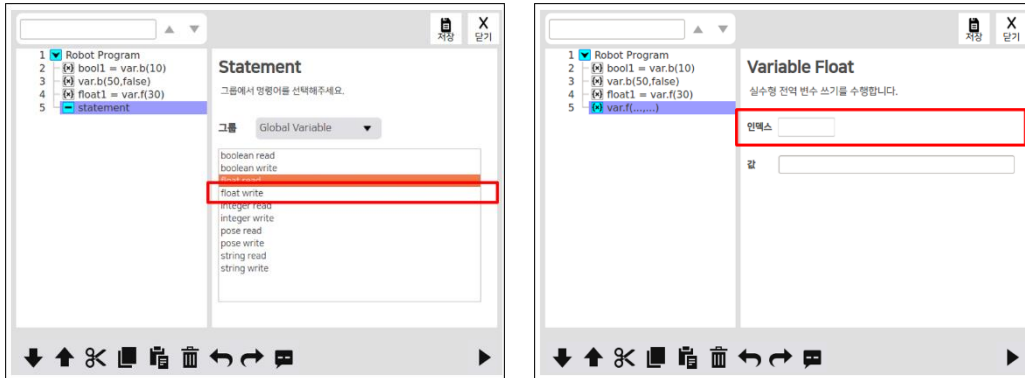


- 1~2000 까지의 실수형 변수 중에 읽어올 인덱스 값을 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.
- 1~2000 까지의 숫자 중에 변수로 선언되어 있는 경우 (ex. a=20), 인덱스에 변수도 입력 가능합니다.

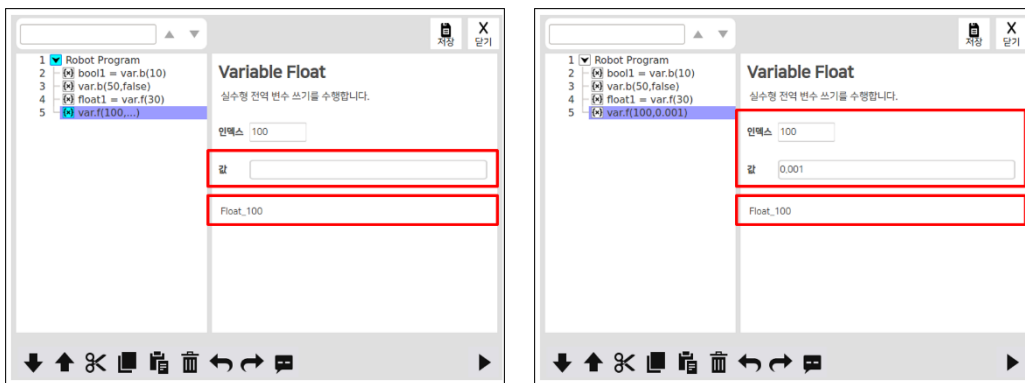


4. Float write

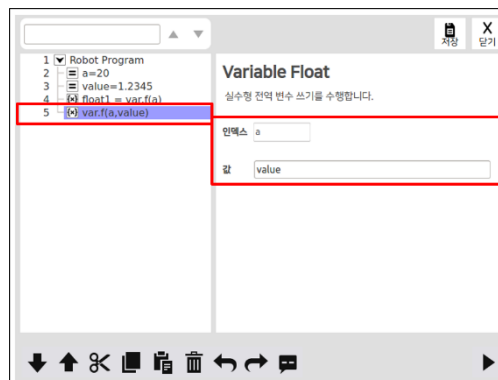
- var.f() 쓰기 명령어는 데이터 메뉴에 있는 실수형 전역 변수를 쓰는 명령어입니다.



- Statement 의 Global Variable 그룹에 있는 float write 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 1~2000 까지의 실수형 변수 중에 읽어올 인덱스 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.

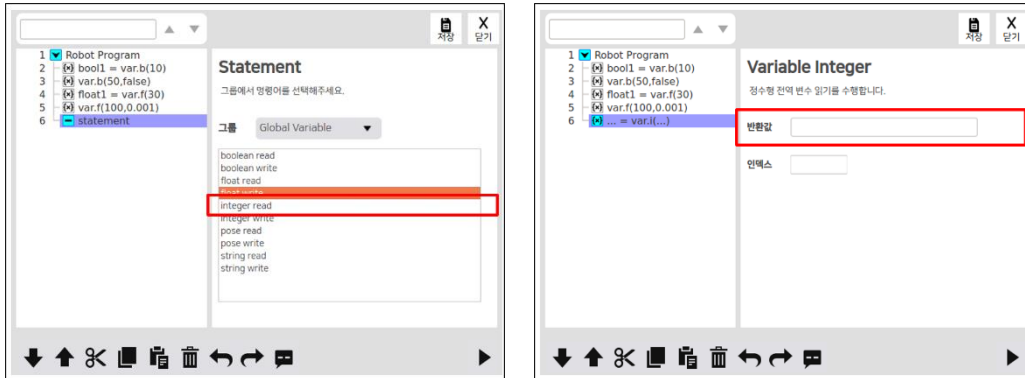


- 값에 실수형 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다.
- 인덱스와 값 모두 선언되어 있는 변수로 입력 가능합니다.

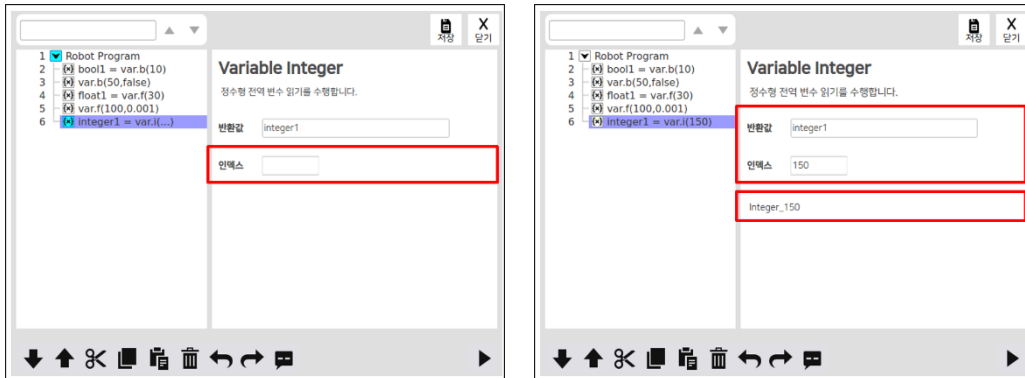


5. Integer read

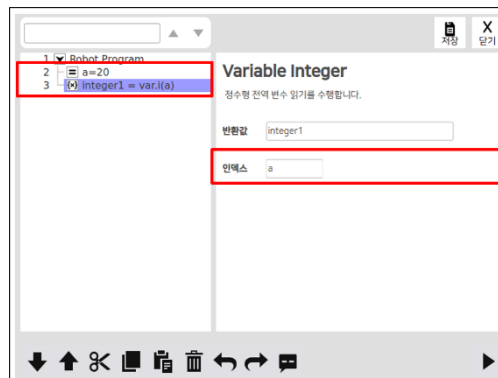
- var.i() 읽기 명령어는 데이터 메뉴에 있는 정수형 전역 변수를 읽어오는 명령어입니다.



- Statement의 Global Variable 그룹에 있는 integer read 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 읽어온 정수형 변수 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

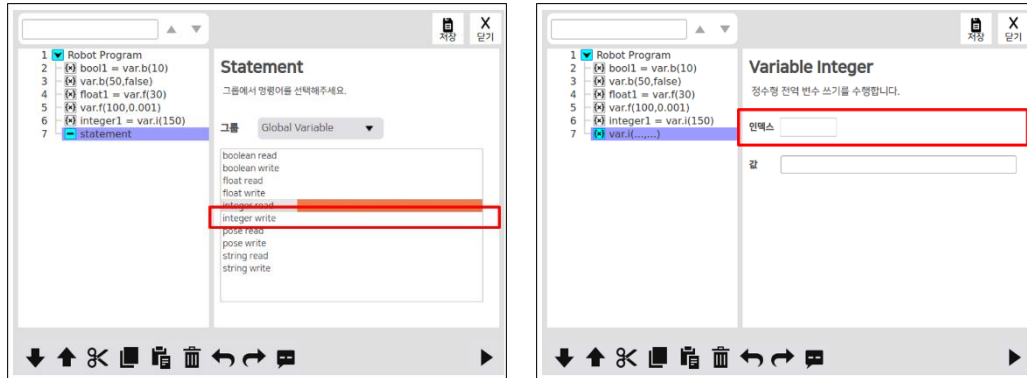


- 1~2000까지의 정수형 변수 중에 읽어올 인덱스 값을 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.
- 1~2000까지의 숫자 중에 변수로 선언되어 있는 경우 (ex. a=20), 인덱스에 변수도 입력 가능합니다.

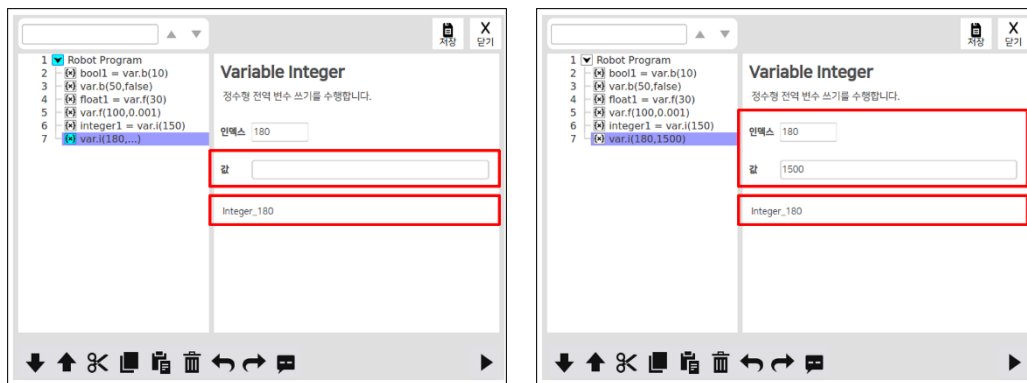


6. Integer write

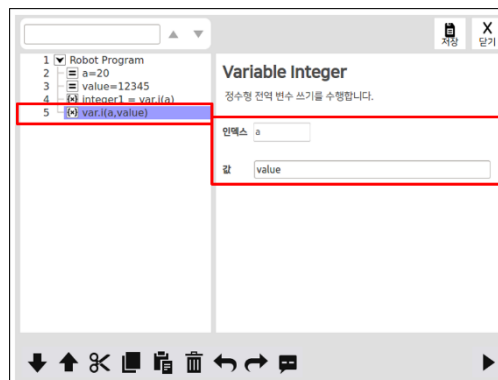
- var.i() 쓰기 명령어는 데이터 메뉴에 있는 정수형 전역 변수를 쓰는 명령어입니다.



- Statement의 Global Variable 그룹에 있는 integer write 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 1~2000까지의 정수형 변수 중에 읽어올 인덱스 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.

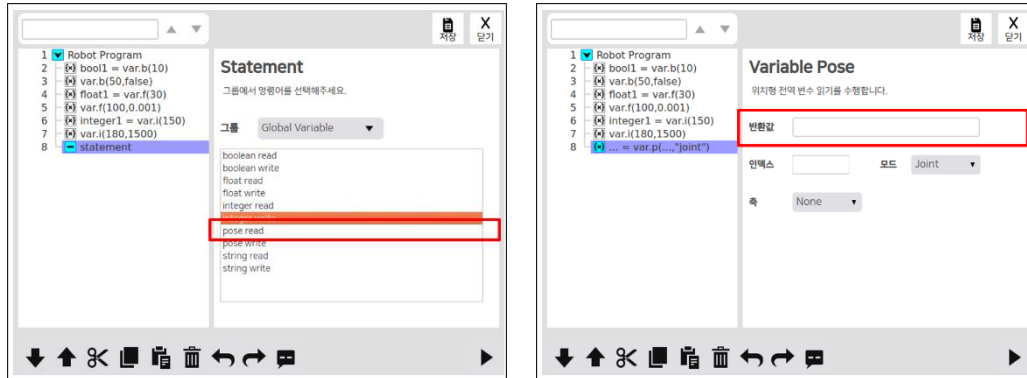


- 값에 정수형 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다.
- 인덱스와 값 모두 선언되어 있는 변수로 입력 가능합니다.

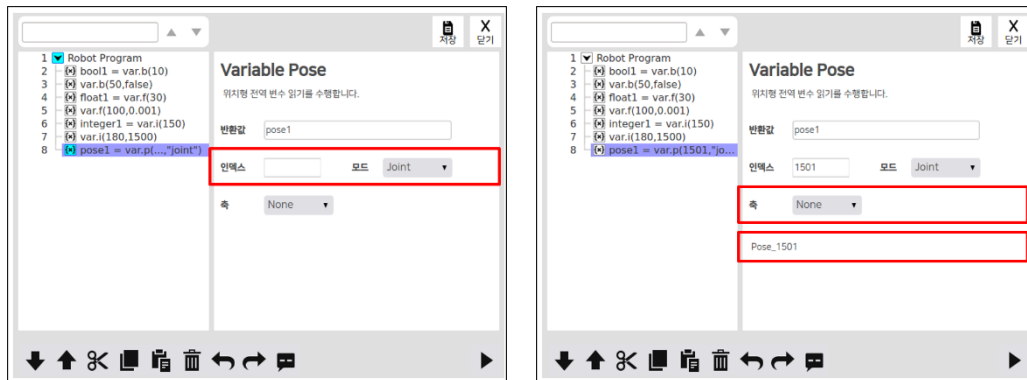


7. Pose read

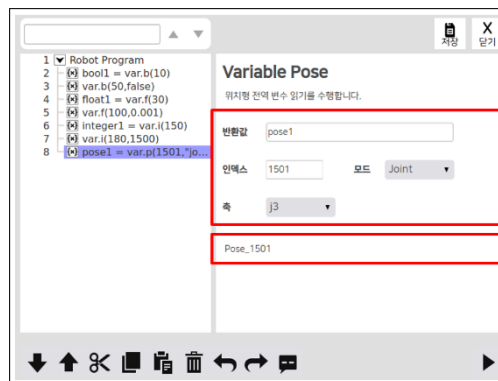
- var.p() 읽기 명령어는 데이터 메뉴에 있는 위치형 전역 변수를 읽어오는 명령어입니다.



- Statement 의 Global Variable 그룹에 있는 pose read 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 읽어온 위치형 변수 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

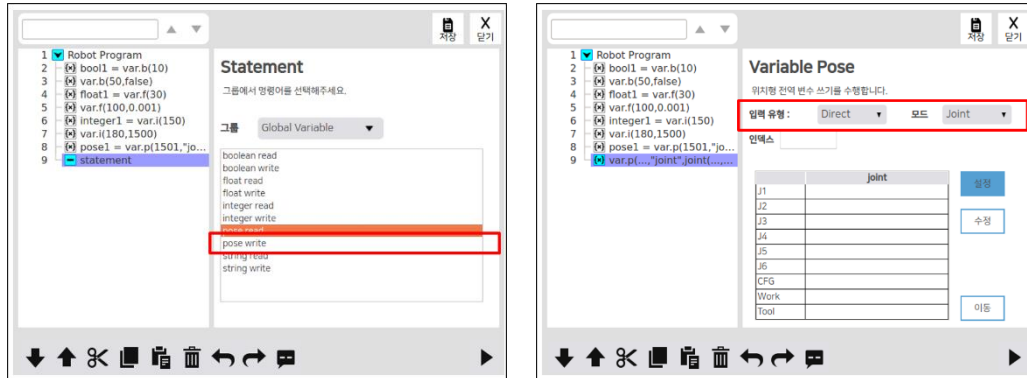


- 1~2000 까지의 논리형 변수 중에 읽어올 인덱스 값을 입력합니다. 1~2000 까지의 숫자 중에 변수로 선언되어 있는 경우 (ex. a=20), 인덱스에 변수도 입력 가능합니다.
- 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.
- 모드는 joint 형과 포즈형을 선택할 수 있습니다.
- 축은 'None'을 선택하면 전 축을 읽어오고 특정 축을 선택하려면 콤보 박스를 통해 선택합니다.

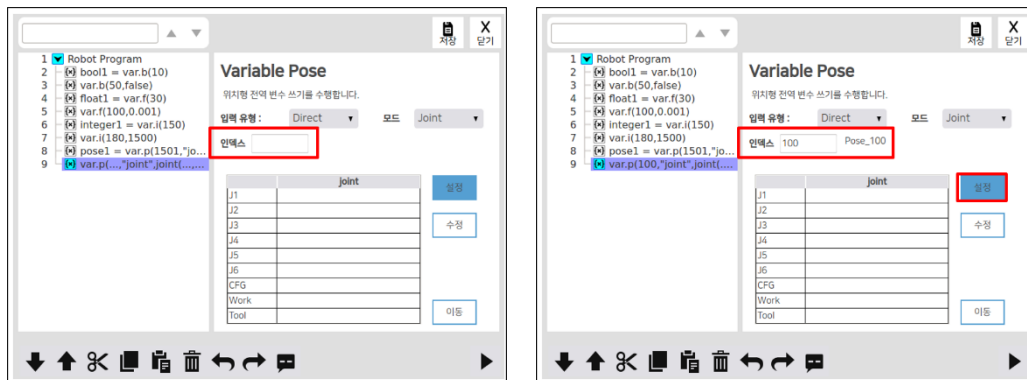


8. Pose write

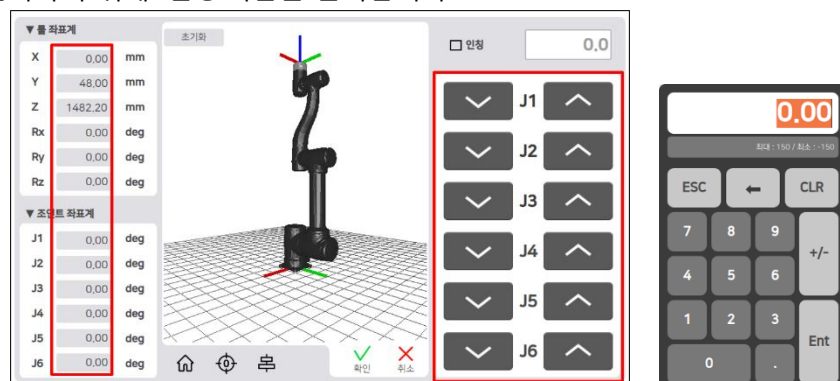
- var.p() 쓰기 명령어는 데이터 메뉴에 있는 위치형 전역 변수를 쓰는 명령어입니다.



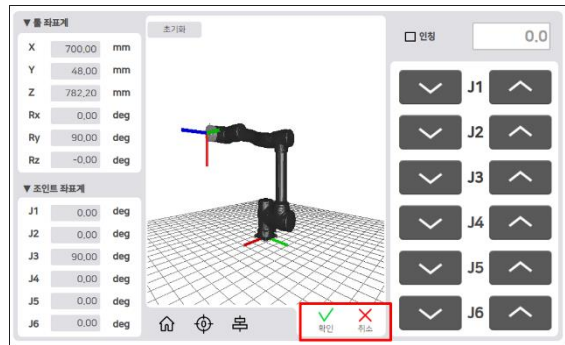
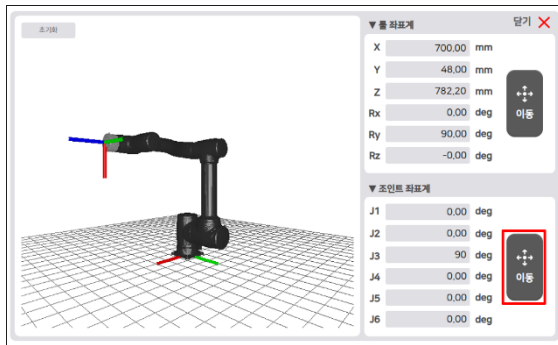
- Statement 의 Global Variable 그룹에 있는 pose write 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 입력 유형은 Direct/Indirect 로, 모드는 Joint/포즈형으로 선택 가능합니다.



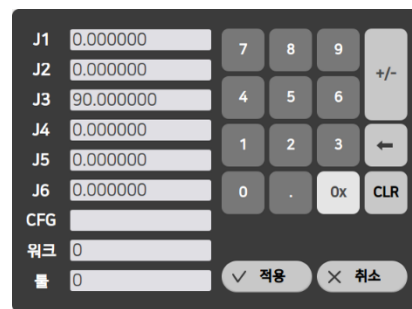
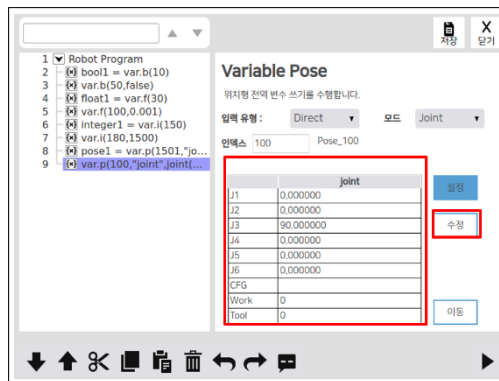
- 1~2000 까지의 위치형 변수 중에 읽어올 인덱스 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 우측에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.
- 인덱스는 선언되어 있는 변수로 입력 가능합니다.
- 위치값을 입력하기 위해 '설정' 버튼을 클릭합니다.



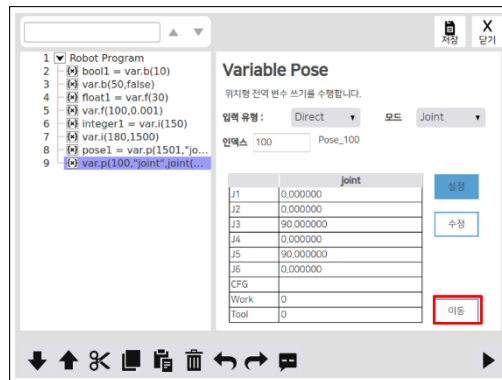
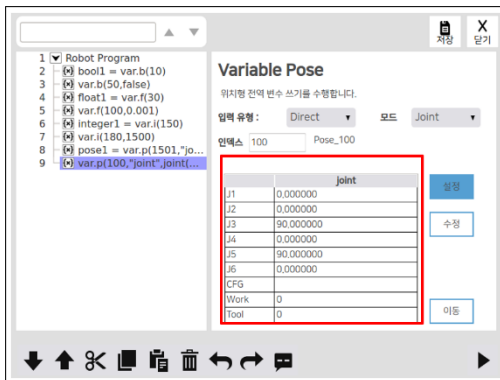
- 조그 화면으로 이동되어 화살표 버튼을 통해 로봇을 이동할 수 있고 톨/조인트 좌표계 칸을 클릭하여 직접 좌표값을 입력하여 이동하는 방법이 있습니다.



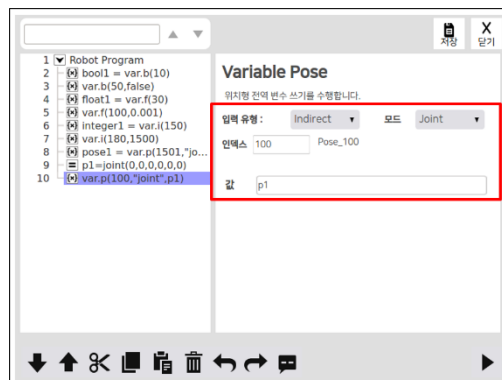
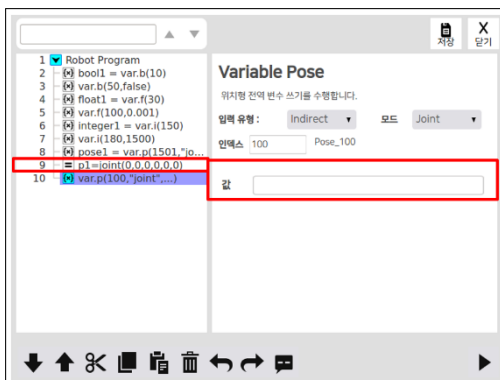
- 좌표값을 입력하고 Enable 키와 '이동'버튼을 통해 로봇을 이동합니다.
- 이동한 좌표값으로 설정하려면 '확인', 설정하지 않으려면 '취소'버튼을 클릭합니다.



- '확인'버튼을 클릭하면 값에 위치형 값이 입력됩니다. '설정'버튼 말고도 '수정'버튼을 클릭하면 값을 입력할 수 있는 키패드가 팝업됩니다.



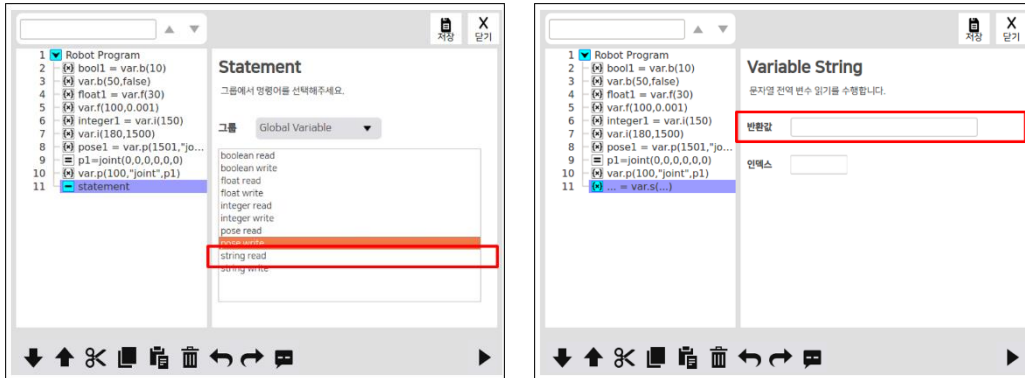
- 값을 입력하고 '적용'버튼을 클릭하면 값 테이블에 설정됩니다.
- 값 테이블에 있는 좌표로 이동하려면 Enable 키와 '이동'버튼을 사용하여 로봇은 이동합니다.



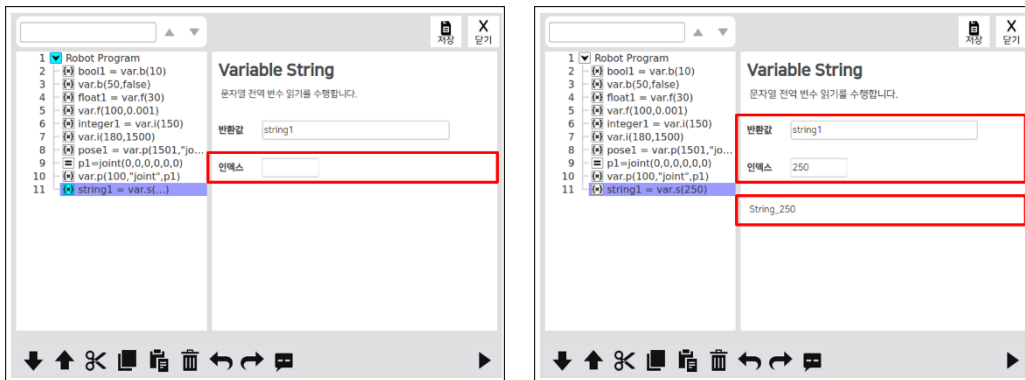
- 입력 유형이 Indirect 면, 값을 직접 입력할 수 있습니다.
- 변수에 좌표를 미리 선언한 후 값 칸에 변수를 입력 할 수 있습니다.

9. String read

- var.s() 읽기 명령어는 데이터 메뉴에 있는 문자열 전역 변수를 읽어오는 명령어입니다.



- Statement 의 Global Variable 그룹에 있는 string read 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 읽어온 문자열 변수 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.

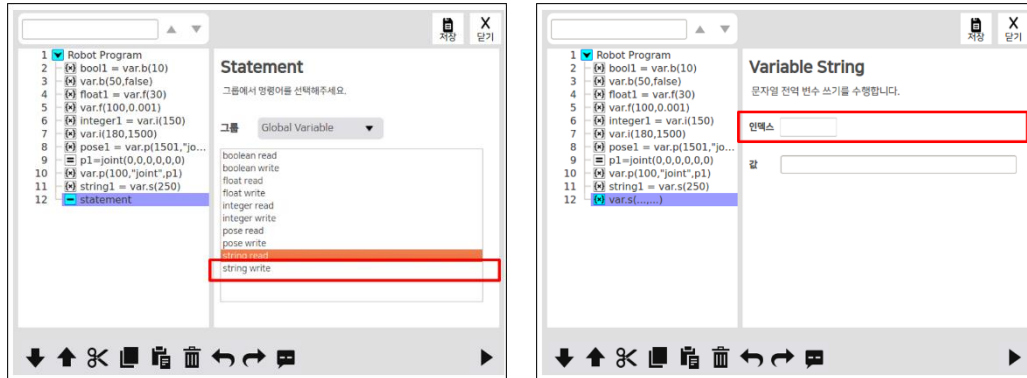


- 1~2000 까지의 문자열 변수 중에 읽어올 인덱스 값을 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.
- 1~2000 까지의 숫자 중에 변수로 선언되어 있는 경우 (ex. a=20), 인덱스에 변수도 입력 가능합니다.

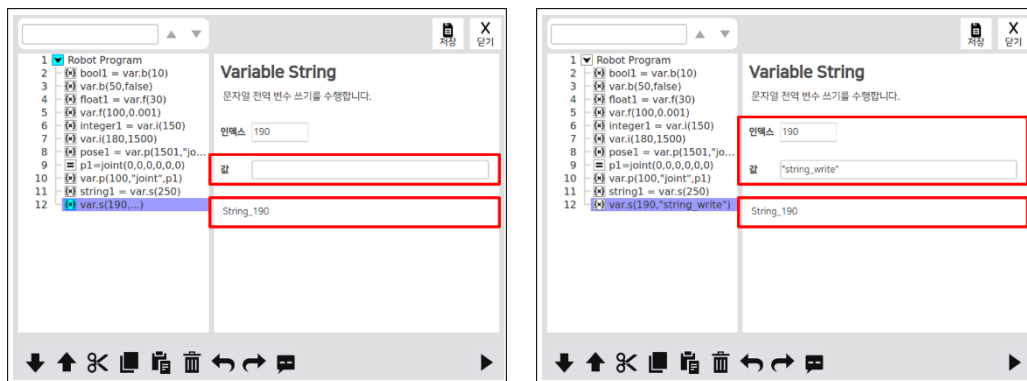


10. String write

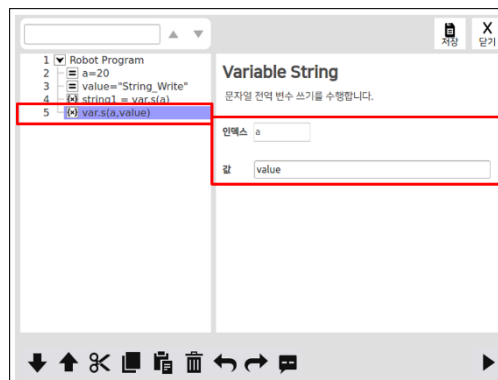
- var.s() 쓰기 명령어는 데이터 메뉴에 있는 문자열 전역 변수를 쓰는 명령어입니다.



- Statement 의 Global Variable 그룹에 있는 string write 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 1~2000 까지의 문자열 변수 중에 읽어올 인덱스 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 하단에 입력 인덱스에 저장되어 있는 설명이 표시됩니다. 입력 인덱스에 설명이 저장되어 있지 않을 경우에는 표시되지 않습니다.



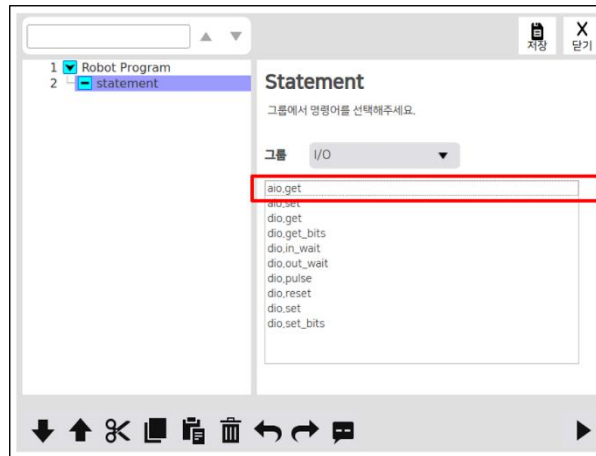
- 값에 문자열 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다.
- 인덱스와 값 모두 선언되어 있는 변수로 입력 가능합니다.



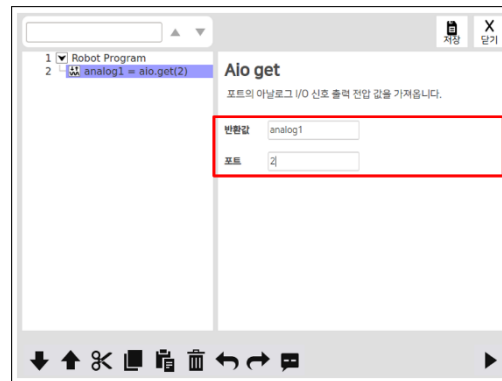
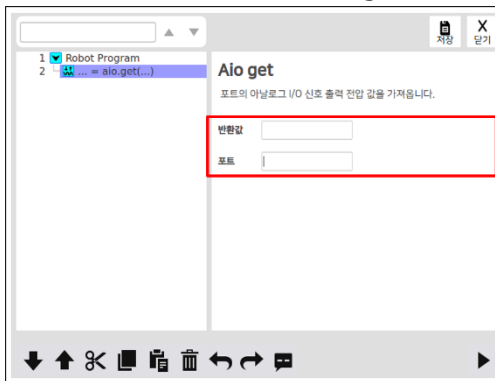
1.1.1.3.3 I/O

1. Aio.get

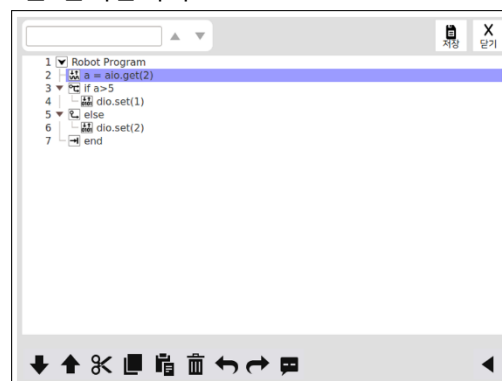
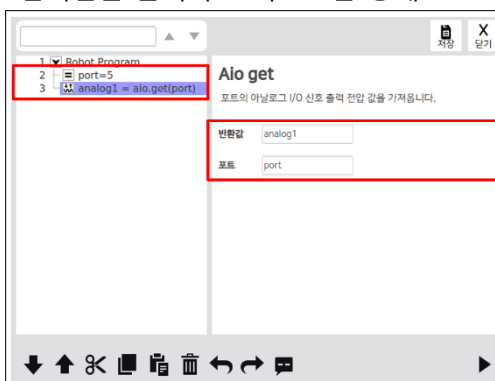
- aio.get 는 입력 포트의 아날로그 I/O 신호 출력 전압 값을 가져오는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 aio.get 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



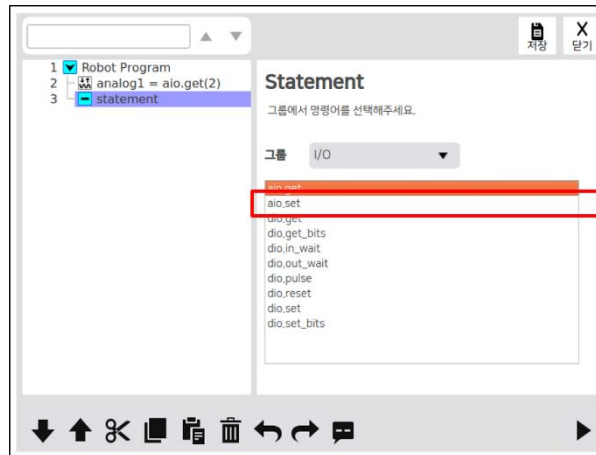
- 변환값은 읽어온 입력 포트의 아날로그 신호 출력 전압 값을 나타내는 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.
- 포트 입력칸을 클릭하고 키보드를 통해 포트 번호를 입력합니다.



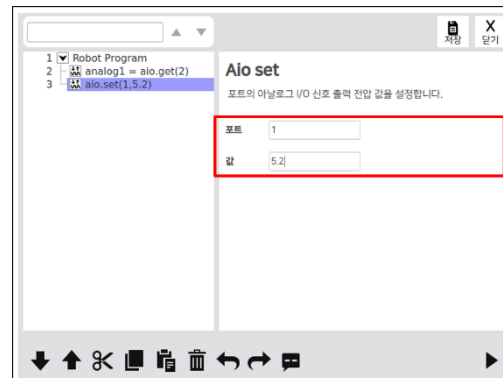
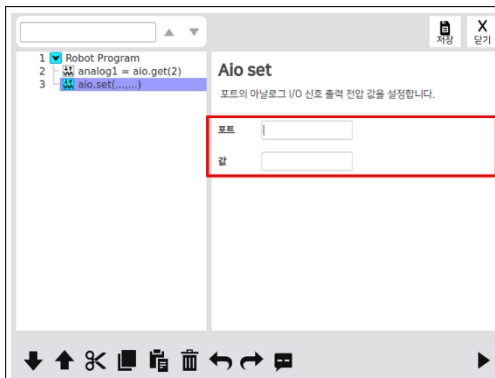
- 포트 번호가 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- aio.get 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

2. Aio.set

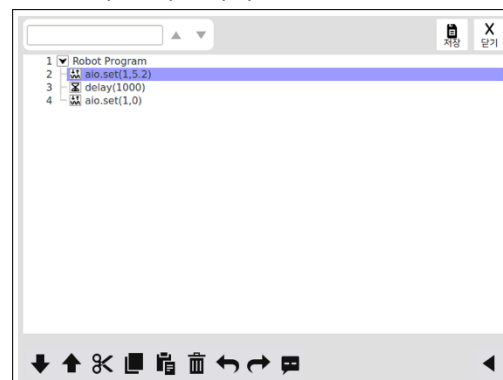
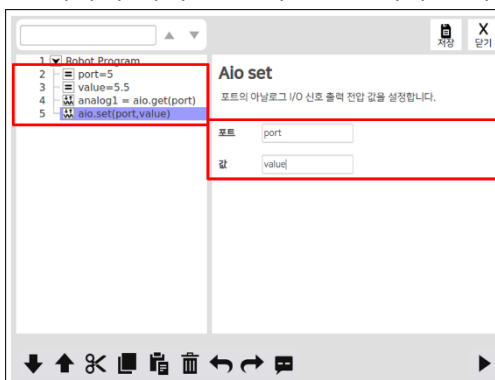
- aio.set 은 입력 포트의 아날로그 I/O 신호 출력 전압 값을 설정하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 aio.set 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



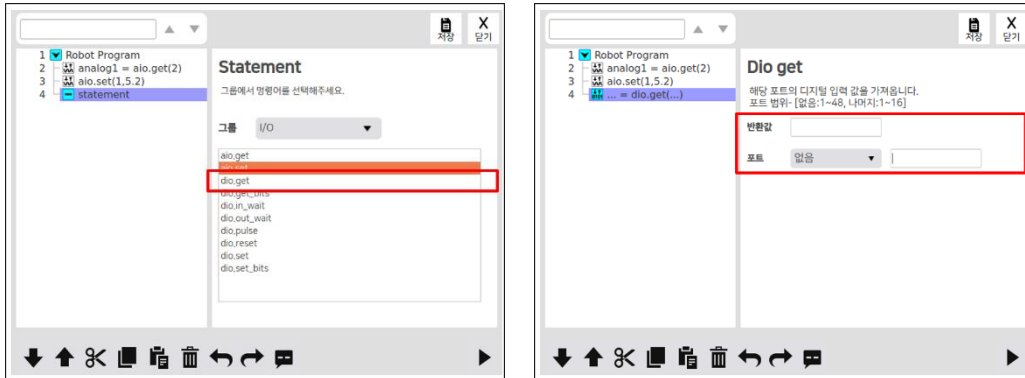
- 전압 값을 설정할 포트 번호를 입력칸을 클릭하고 키보드를 통해 입력합니다.
- 값을 입력하기 위해 값 입력칸을 클릭하고 키보드를 통해 입력합니다.



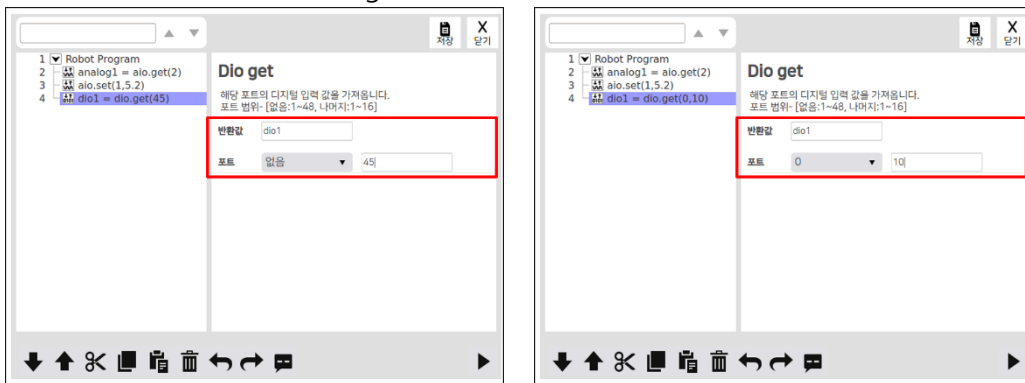
- 포트 번호와 값이 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- aio.set 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

3. Dio.get

- dio.get 은 입력 포트의 디지털 입력 값을 가져오는 명령어 입니다.



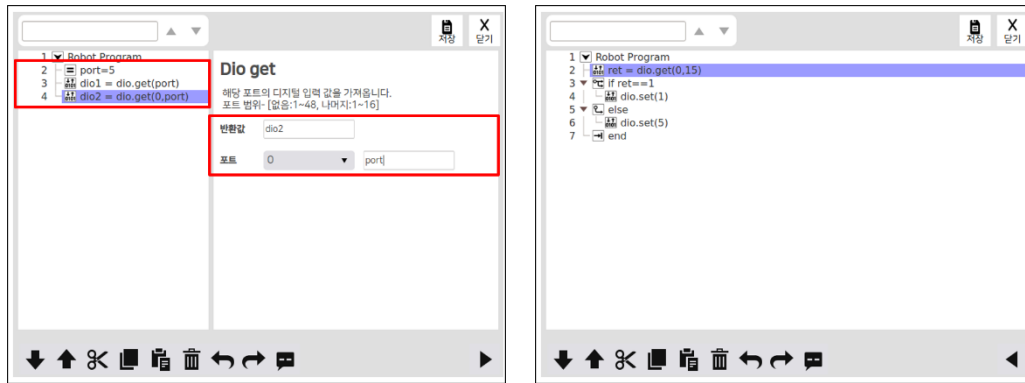
- Statement 의 I/O 그룹에 있는 dio.get 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- 반환값은 읽어온 입력 포트의 디지털 입력 값을 나타내는 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.
- 포트의 콤보 박스는 모듈을 선택할 수 있습니다. 콤보박스로 모듈을 선택하고 우측 입력칸을 클릭하고 키보드를 통해 포트 번호를 입력합니다.

모듈	포트범위
없음	1~48
0	1~16
1	
2	

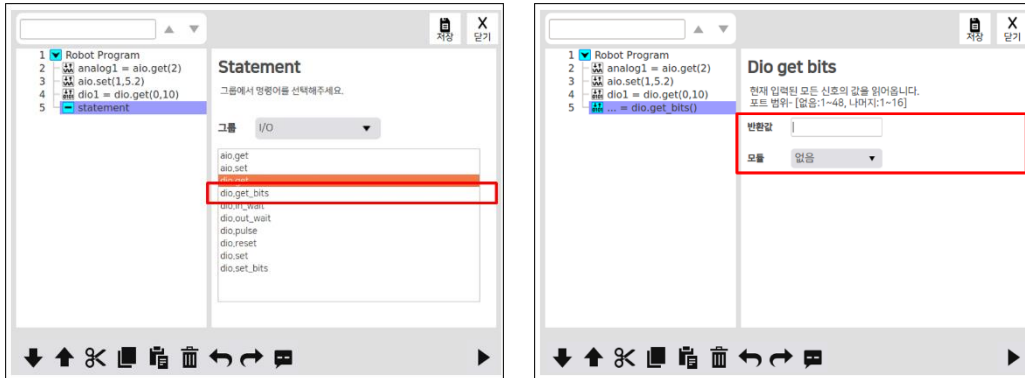
- 모듈은 확장 IO 모듈을 선택할 수 있습니다. '없음'일 때는 전 포트에서 포트를 설정할 수 있고 0, 1, 2 모듈은 각 16 개의 포트 중에 포트를 설정할 수 있습니다. 1, 2 모듈은 확장 IO 모듈이 설치되어 있을 때만 표시됩니다.



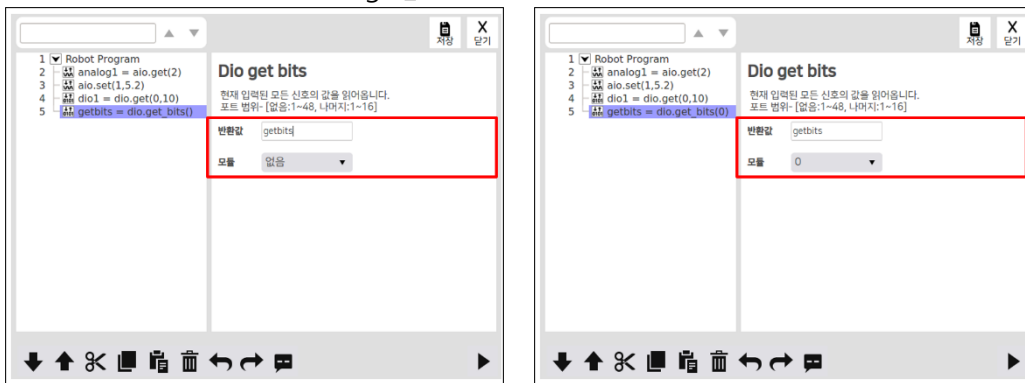
- 포트 번호가 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.get 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

4. Dio.get_bits

- dio.get_bits 는 현재 입력된 모든 신호의 값을 읽어오는 명령어 입니다.



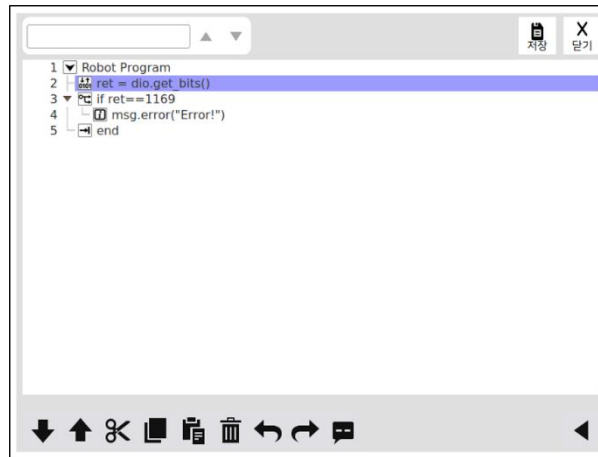
- Statement 의 I/O 그룹에 있는 dio.get_bits 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- 반환값은 현재 입력된 모든 신호의 값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.
- 모듈은 콤보 박스를 통해 모듈을 선택할 수 있습니다.

모듈	포트범위
없음	1~48
0	1~16
1	
2	

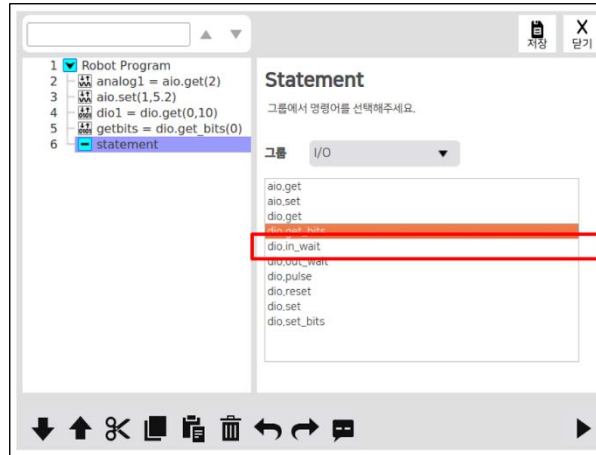
- 모듈은 확장 IO 모듈을 선택할 수 있습니다. '없음'일 때는 전 포트의 신호 값을 읽어오고 0, 1, 2 모듈은 설정한 모듈의 신호 값을 읽어옵니다. 1, 2 모듈은 확장 IO 모듈이 설치되어 있을 때만 표시됩니다.



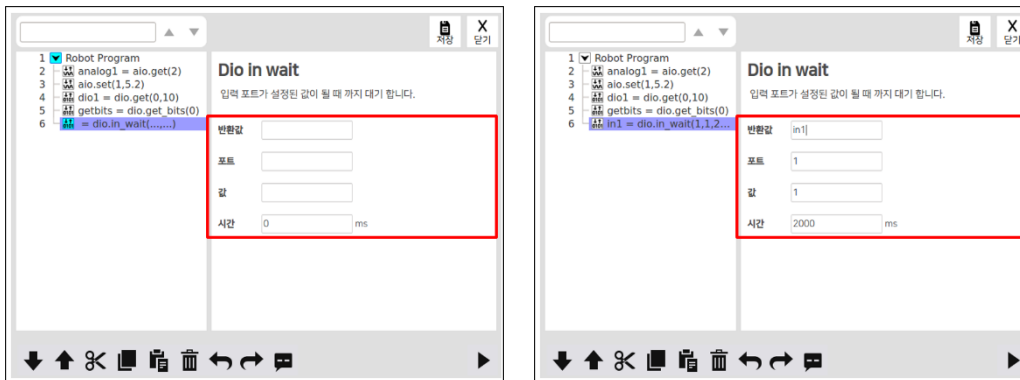
- dio.get_bits 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

5. Dio.in_wait

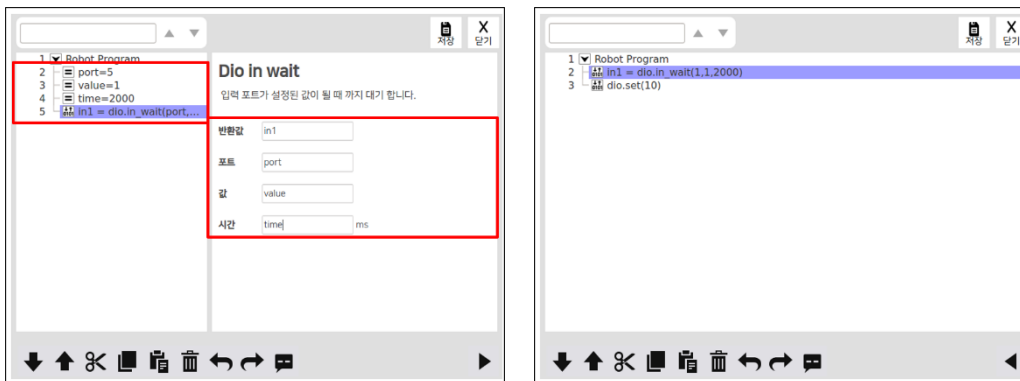
- dio.in_wait 은 입력 포트가 설정된 값이 될 때 까지 대기하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 dio.in_wait 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



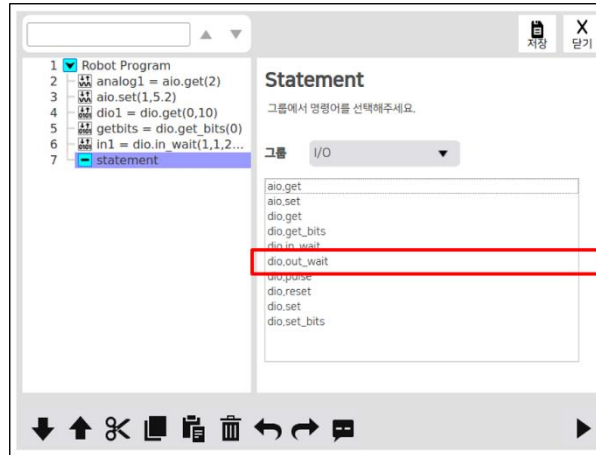
- 반환값, 포트번호, 값과 시간을 입력하기 위해 각 입력칸을 클릭하고 키보드를 통해 입력합니다.
- 시간은 설정하지 않아도 디폴트로 0ms 로 설정됩니다.



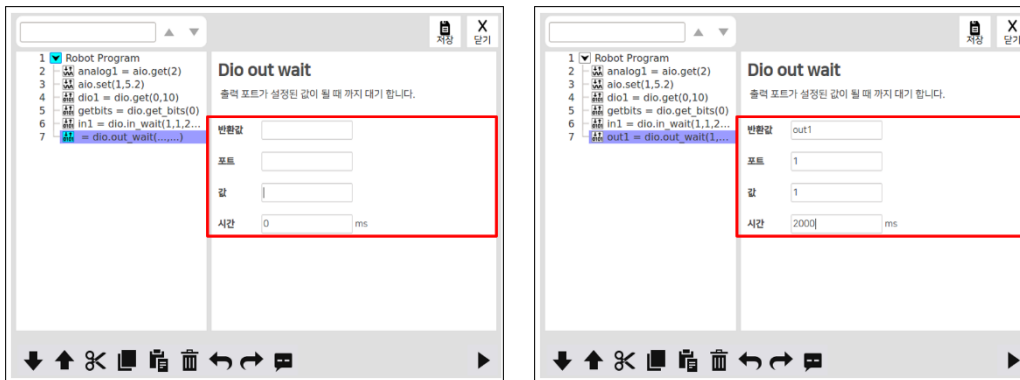
- 포트 번호와 값, 시간이 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.in_wait 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

6. Dio.out_wait

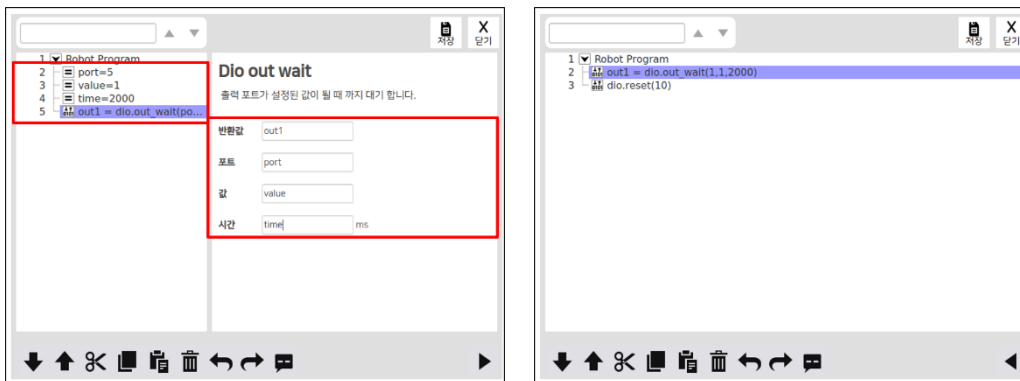
- dio.out_wait 은 입력한 출력 포트가 설정된 값이 될 때 까지 대기하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 dio.out_wait 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



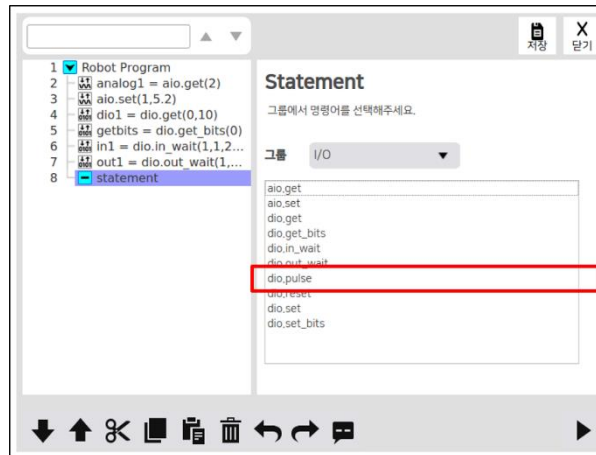
- 반환값, 포트번호, 값과 시간을 입력하기 위해 각 입력칸을 클릭하고 키보드를 통해 입력합니다.
- 시간은 설정하지 않아도 디폴트로 0ms 로 설정됩니다.



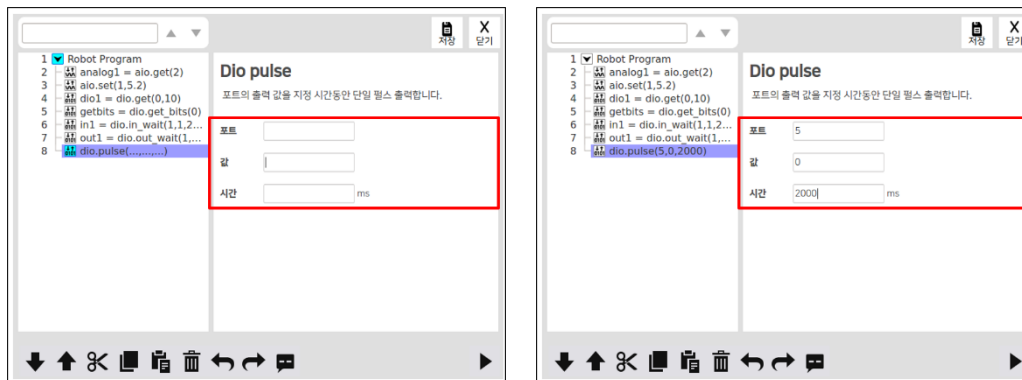
- 포트 번호와 값, 시간이 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.out_wait 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

7. Dio.pulse

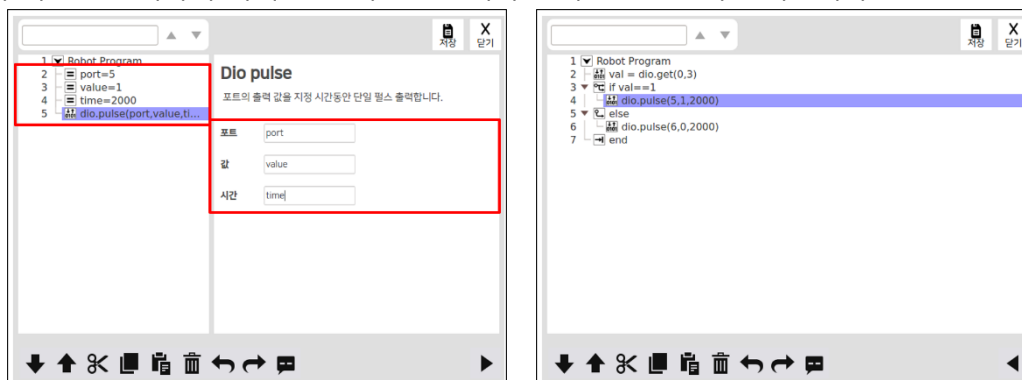
- dio.pulse 는 입력한 포트의 출력 값을 지정 시간 동안 단일 펄스를 출력하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 dio.pulse 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 포트 번호를 입력칸을 클릭하고 키보드를 통해 입력합니다.



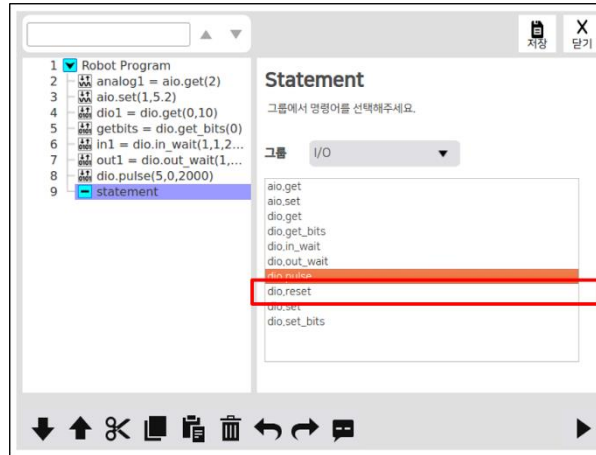
- 값과 시간을 입력하기 위해 값 입력칸을 클릭하고 키보드를 통해 입력합니다.



- 포트 번호와 값, 시간이 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.pulse 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

8. Dio.reset

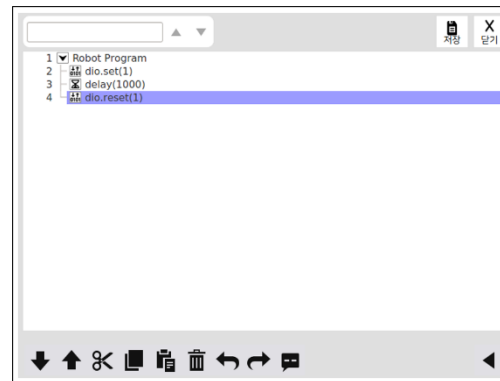
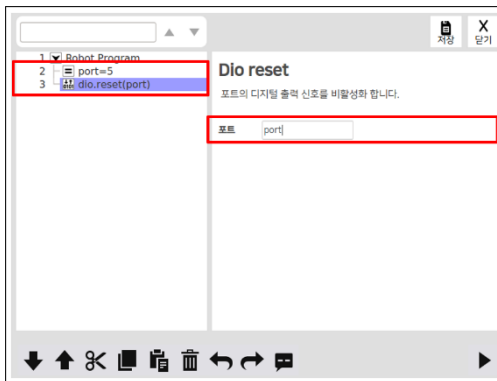
- dio.reset 은 입력한 포트의 디지털 출력 신호를 비활성화하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 dio.reset 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



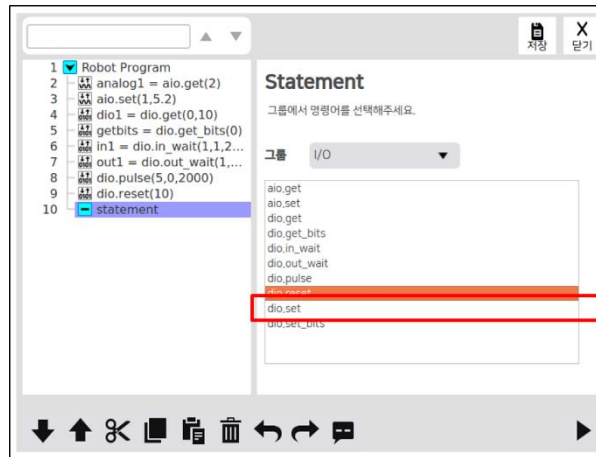
- 포트 번호를 입력칸을 클릭하고 키보드를 통해 입력합니다.



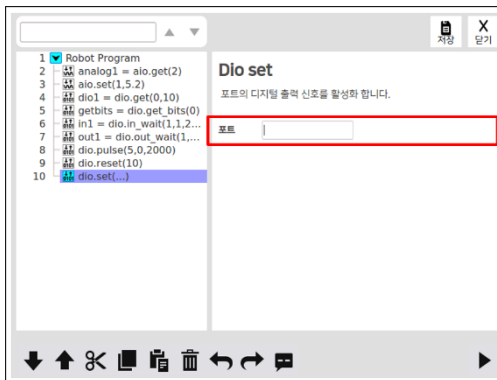
- 포트 번호가 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.reset 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

9. Dio.set

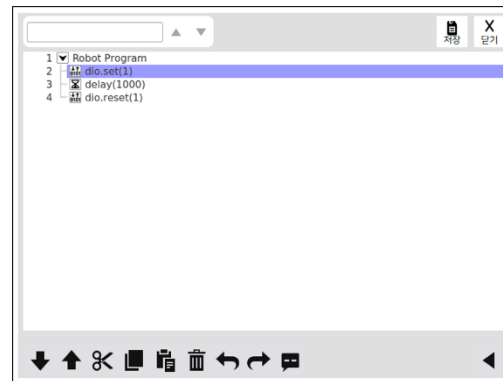
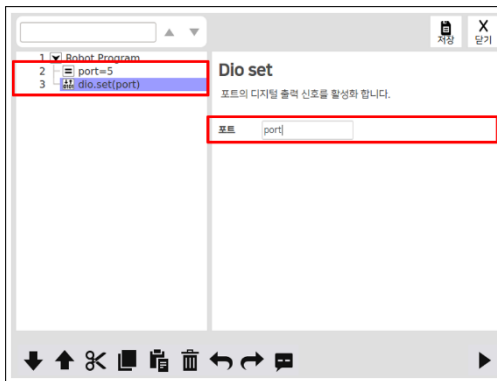
- dio.set 은 입력한 포트의 디지털 출력 신호를 활성화하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 dio.set 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



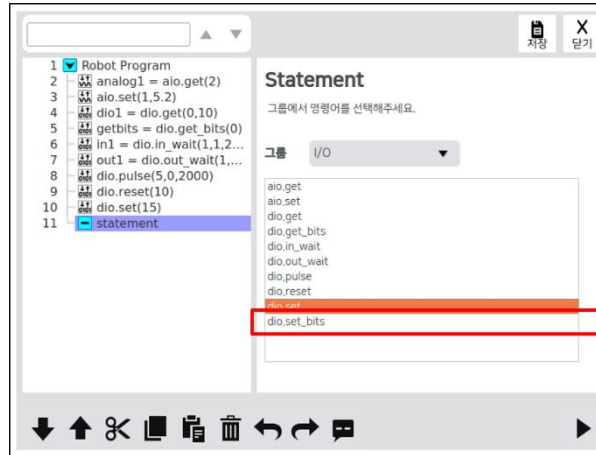
- 포트 번호를 입력칸을 클릭하고 키보드를 통해 입력합니다.



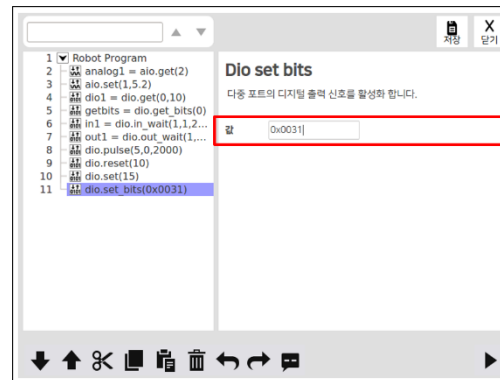
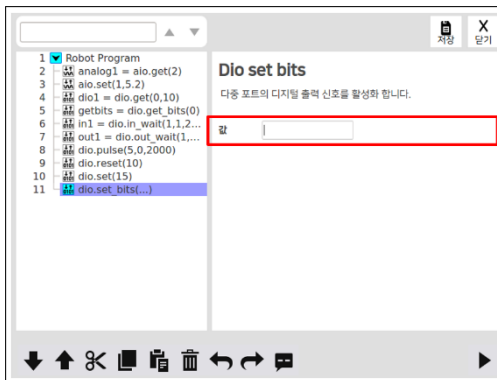
- 포트 번호가 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.set 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

10. Dio.set_bits

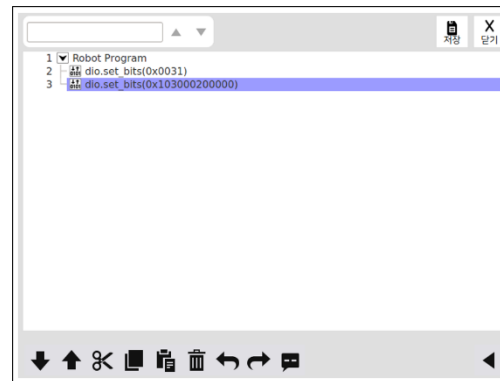
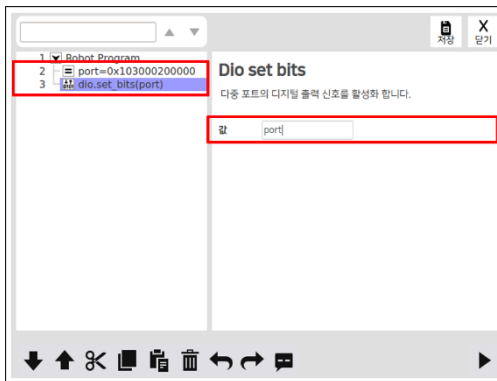
- dio.set_bits 는 다중 포트의 디지털 출력 신호를 활성화 하는 명령어 입니다.



- Statement 의 I/O 그룹에 있는 dio.set_bits 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.



- 포트 번호를 입력칸을 클릭하고 키보드를 통해 입력합니다.

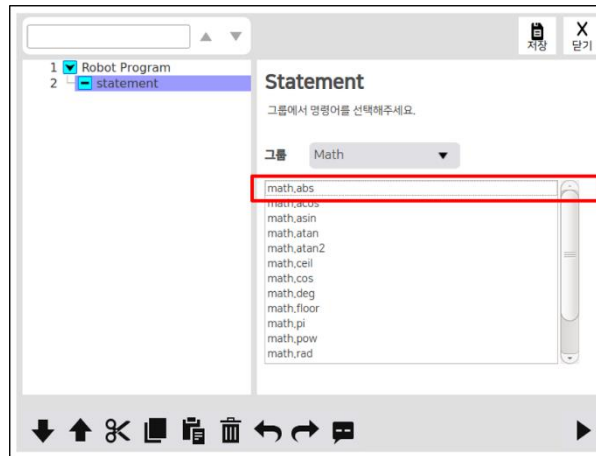


- 포트 번호가 변수로 선언되어 있는 경우, 변수 입력이 가능합니다.
- dio.set_bits 명령어 예시 입니다. 트리뷰 프로그램 작성에 참고 바랍니다.

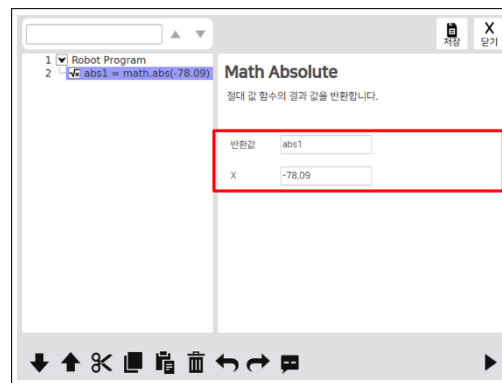
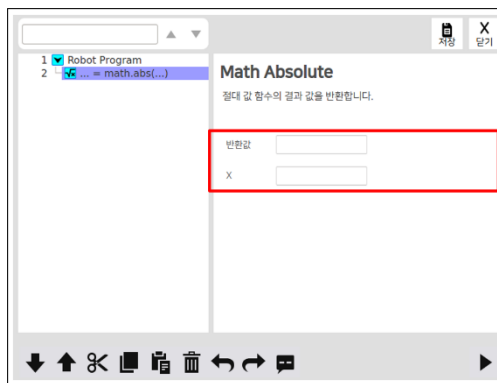
1.1.1.3.4 Math

1. Abs

- math.abs 는 절대 값 함수의 결과 값을 반환하는 명령어 입니다.



- Statement 의 Math 그룹에 있는 math.abs 명령어를 클릭하여 트리뷰에 추가하여 사용합니다.
- 반환값은 입력한 X 값의 절대값을 저장할 변수입니다. 입력칸을 클릭하고 키보드를 통해 입력합니다.



- X 칸에 절대값으로 반환할 값을 입력합니다. 칸을 클릭하고 키보드를 통해 입력합니다. 이 X 값은 선언되어 있는 변수로 입력 가능합니다.

